

provided, for example the ‘less than’ relationship between numbers, and the ‘addition’ function. A LISP-like notation is also provided for handling lists of objects. Using this basic apparatus, it is possible to *define* new objects, and the functional and other relationships between these objects. At this point, some examples seem appropriate. The following KIF expression asserts that the temperature of m1 is 83 Celsius:

```
(= (temperature m1) (scalar 83 Celsius))
```

In this expression, = is equality: a relation between two objects in the domain; temperature is a function that takes a single argument, an object in the domain (in this case, m1), and scalar is a function that takes two arguments. The = relation is provided as standard in KIF, but both the temperature and scalar functions must be defined.

The second example shows how definitions can be used to introduce new concepts for the domain, in terms of existing concepts. It says that an object is a bachelor if this object is a man and is not married:

```
(defrelation bachelor (?x) :=  
  (and (man ?x)  
        (not (married ?x))))
```

In this example, ?x is a variable, rather like a parameter in a programming language. There are two relations: man and married, each of which takes a single argument. The := symbol means ‘is, by definition’.

The next example shows how relationships between individuals in the domain can be stated – it says that any individual with the property of being a person also has the property of being a mammal:

```
(defrelation person (?x) :=> (mammal ?x))
```

Here, both person and mammal are relations that take a single argument.

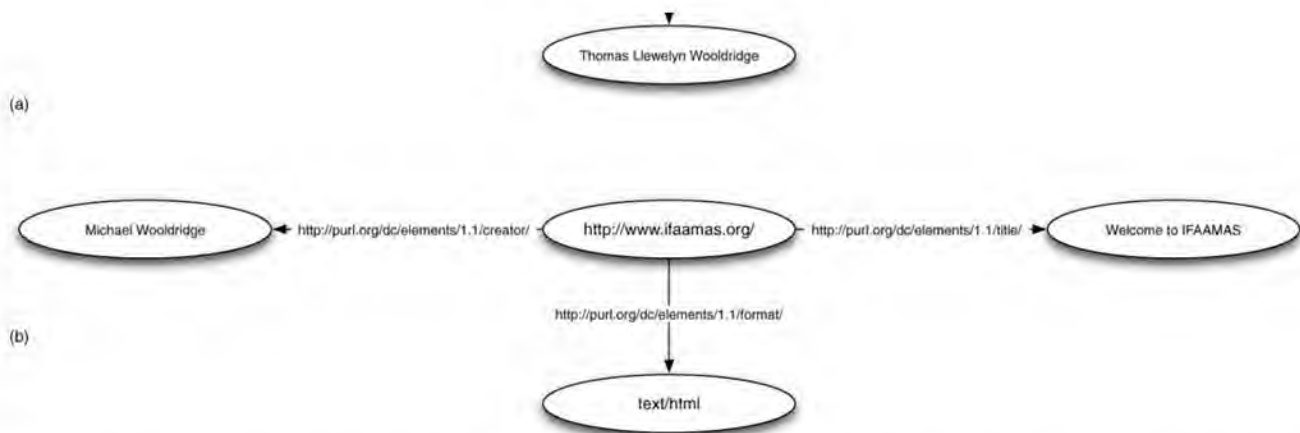
6.3 RDF

RDF

The *resource definition framework* (RDF) is not an ontology language, but it is very closely connected with ontologies in general and the semantic web in particular, and it seems appropriate to discuss it here. Essentially, RDF was developed with the goal of providing a *standardised knowledge representation framework for the web*. In terms of its properties as a knowledge representation formalism, RDF is very simple, which is both a disadvantage and an advantage: it is a disadvantage because it means that many things are hard to represent using RDF, but it is an advantage because it makes the framework comparatively tractable (at least compared with frameworks like OWL).

Figure 6.7: Some semantic networks: (a) is an informal example of family relationships, while (b) is an example of the kind of knowledge RDF is specifically intended to represent.





RDF is basically a language for representing *subject–predicate–object* triples. The notion of an RDF triple is best explained by example. [Figure 6.7a](#) shows a small *semantic net*, which captures some information about my family relationships. The meaning of the network is, I hope, clear: the ovals represent entities, an arc from one oval to another indicates a relationship between the entities, and the label on the arc indicates the nature of the relationship [Russell and Norvig, [1995](#)]. Semantic nets like this can easily be seen to correspond to a simple fragment of first-order logic; for example, we can immediately translate [Figure 6.7a](#) into the following facts in first-order logic:

RDF TRIPLES

isMarriedto(Michael, Janine)

isFatherOf(Michael, Tom)

isFatherOf(Michael, Lily).

RESOURCES

Each of these facts corresponds exactly to an RDF triple. In the first fact, the *subject* is Michael, the *predicate* is ‘isMarriedTo’, and the *object* is Janine. An RDF document is simply a set of such triples, and it is in this way that an RDF document can be understood as a knowledge base. However, RDF is intended specifically for representing knowledge in web applications, and so the actual format of RDF documents is based on XML, and the subjects referred to in a triple are typically *resources*. References to resources are defined using the *uniform resource identifier* (URI). The best-known examples of URIs are URLs: strings of the form `http://xxx.yyy/` that we use to identify and navigate to web pages. URLs are just one example of URIs, and by using URIs we can identify many kinds of network-accessible resources, although for the purposes of this section, it is enough to consider URLs. Consider the semantic net in [Figure 6.7b](#) (the network structure is deliberately identical to that of [Figure 6.7\(a\)](#)). The network makes three assertions about the web resource <http://www.ifaamas.org/>, namely:

URI

- the creator of the resource is Michael Wooldridge
- the title of the resource is ‘Welcome to IFAAMAS’
- the format of the resource is text/html.

For this information to be processable by a machine, however, simple textual labels such as ‘creator’ and ‘title’ for predicates are no use; we need to somehow use an unambiguous computer-processable definition of these predicates. The idea in RDF is to make these predicates resources as well – in particular, to make them web resources. Thus for example, the following URL is used to define the ‘creator’ predicate:

<http://purl.org/dc/elements/1.1/creator/>

If you followed this reference, you would find an RDF definition of ‘creator’. This definition serves as a precise definition of the predicate. Nodes in RDF semantic nets can be resources, literals (e.g. the string ‘Michael Wooldridge’) or may be *blank*. Blank nodes are essentially ‘dummy’ nodes – they are used basically because RDF is limited to binary relationships, i.e. things like *isFatherOf(Michael, Tom)*.

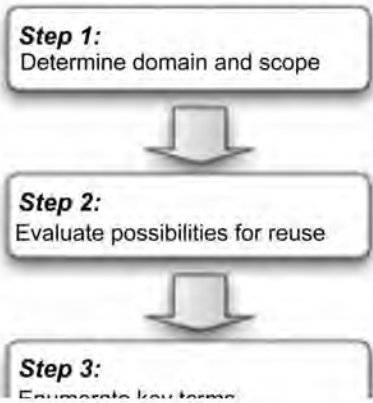
BLANK NODE

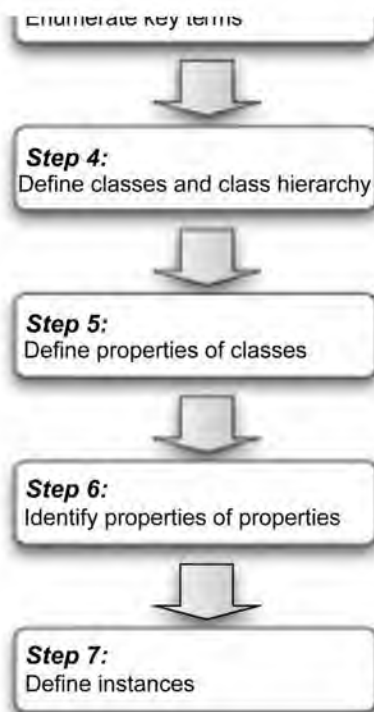
```
<?xml version="1.0" ?>
<rdf:RDF
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:dc="http://purl.org/dc/elements/1.1/">
  <rdf:Description about="http://www.ifaamas.org">
    <dc:title>Welcome to IFAAMAS</dc:title>
    <dc:author>Michael Wooldridge</dc:date>
    <dc:format>text/html</dc:format>
  </rdf:Description>
</rdf:RDF>
```

Here, the first tag identifies the XML version used; the next tag defines the *namespaces* used in the document: in this case, we use the RDF namespace from the worldwide web consortium (‘XMLNS:RDF’ = ‘XML namespace: RDF’), and the ‘Dublin core’ namespace, which is concerned with document metadata (the attributes of a document such as author, language, publication date, etc. – thus ‘XMLNS:DC’ = ‘XML namespace: Dublin core’). The final tag defines the triples themselves: each triple is ‘about’ a single resource, so we only have to list the resource once.

NAMESPACE

Figure 6.8: Ontology development 101: the ontology development methodology of Noy and McGuinness.





6.4 Constructing an Ontology

Now that we have considered several languages for defining ontologies, let us move on to consider the issue of how to go about constructing an ontology. As with languages, there are many different methodologies. Here, we will consider the general methodology proposed in [Noy and McGuinness, 2004]. You should note, however, that it is entirely possible to follow the methodology to the letter, but still end up with a poor ontology: the methodology simply provides guidelines to structure your approach. The main steps in the methodology are as follows (see [Figure 6.8](#)).

Step 1: Determine domain and scope

If you have ever studied software engineering, you will know that the first step in any software engineering methodology is that of understanding and formulating *requirements*: you can think of the first step in ontology development as being basically the same. Thus the first step in ontology development involves answering questions such as [Noy and McGuinness, 2004]:

- What domain will my ontology cover?
- What questions will my ontology be used to answer?
- How will the ontology be used?

Step 2: Consider reuse

Reuse is extremely important in ontology development, for the simple reason that an ontology is most useful if everybody uses it: if we all define our own ontology for every application, then we are defeating the object of sharing meaning. Many libraries of predefined ontologies are freely available on the web, or are available for purchase. For example, the SNOMED ontology defines a huge range of clinical terms, and has been refined and extended over many years [Spackman et al., 1997]. The Cyc ontology attempts to provide a rigorous formulation of ‘human consensus reality’, that is, the world as humans perceive and understand it [Cycorp, 2008].

Step 3: Enumerate all the relevant terms

This step simply involves brainstorming all the terms associated with our domain literally, listing all the domain-specific words and concepts that appear in our requirements. For example, with respect to [Figure 6.1](#), we might list ‘book’, ‘author’, ‘work of art’, ‘publisher’, and so on. Notice that, at this stage, we are not trying to *organize* these terms in any specific way – in particular, we are not attempting to identify classes, properties, and so on. We are simply trying to identify everything that might be included, somehow, in the ontology.

Step 4: Define classes and class hierarchy

Next, we begin to organize our understanding of the domain, by beginning to identify classes and organize them. A common mistake when doing this is to confuse classes and their properties. For example, suppose you identified the terms ‘person’ and ‘age’ as being relevant for your domain. It would not be *wrong* to have both of these as classes, but perhaps it makes more sense for ‘person’ to be a class and ‘age’ to be a property (in this case, a positive number). Another common mistake is to have ‘trivial’ classes. For example, it probably does not make much sense to have a class for ‘Person with brown hair and blue eyes’. It would perhaps make sense to have a ‘person’ class, and ‘hair colour’ and ‘eye colour’ as properties.

Refining classes into hierarchies can be done either *top down* (identifying the most general classes, then the next most general, and so on), or else *bottom up* (clustering similar terms into progressively more general classes).

TOP-DOWN REFINEMENT

BOTTOM-UP REFINEMENT

It is worth again emphasizing that there will in general be many ways of organizing a group of terms into an ontology, and these different organizations will each have their own merits. It is also worth bearing in mind that being pedantic may be a disadvantage. Suppose you are building an ontology to help a grocery store organize its produce displays. Now consider a tomato. Tomatoes are, scientifically, fruits, rather than vegetables. But if your ontology classifies them as fruit, you may well find that this confuses customers, who will likely think of them as vegetables, and expect to find them in the vegetable section of the shop. Here, the scientific classification is at odds with the intended purpose of the ontology.

Step 5: Define properties

This step involves identifying, for each class, the properties that are associated with that class. We attach these properties to the most general class that has them. Properties come in several different types [Noy and McGuinness, [2004](#)]:

- *Intrinsic properties* are those that relate to the nature of an object – for example, they may be measurable properties such as weight, height, and so on.

INTRINSIC PROPERTIES

- *Extrinsic properties* are abstract properties such as ‘name’ ‘social security number’

Extrinsic properties are abstract properties such as name, social security number, and so on, which are attached to an object. Typically, we would not be able to derive these properties simply by examining an object.

EXTRINSIC PROPERTIES

- *Components of an object* If the object is structured in some way, then it might be useful to identify its component parts.
- *Relationships* For example, we might have an ‘author’ relationship, linking an object of type ‘novel’ to an object of type ‘person’.

Step 6: Define properties of properties

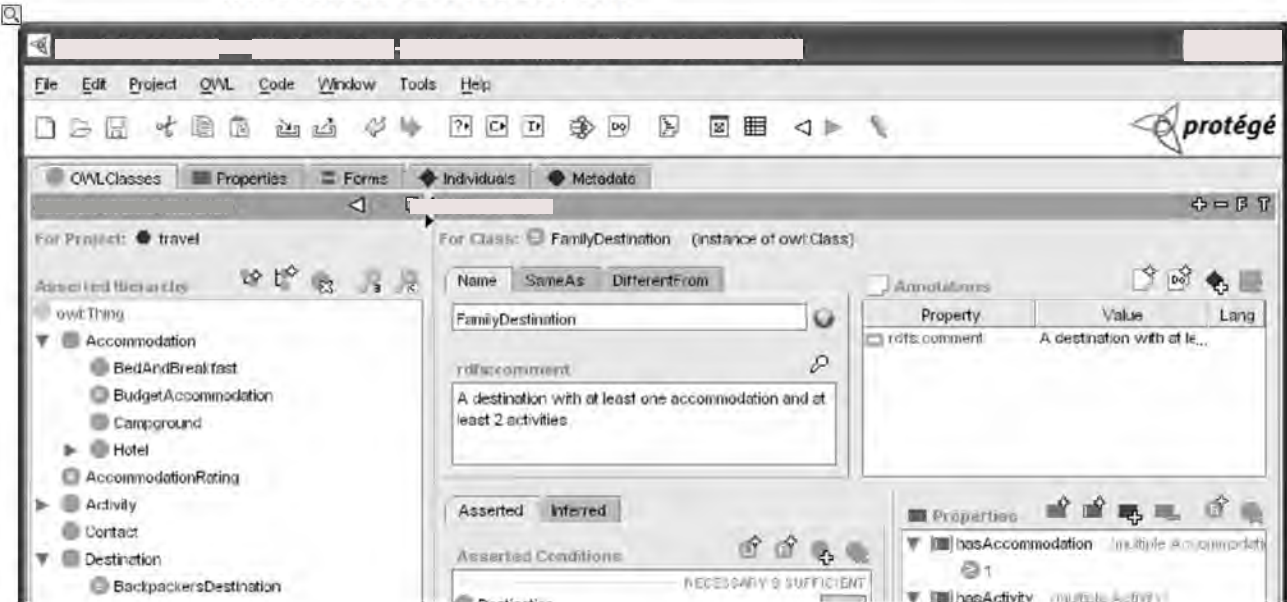
The aim at this stage is to identify the properties that each property has. Example properties might be:

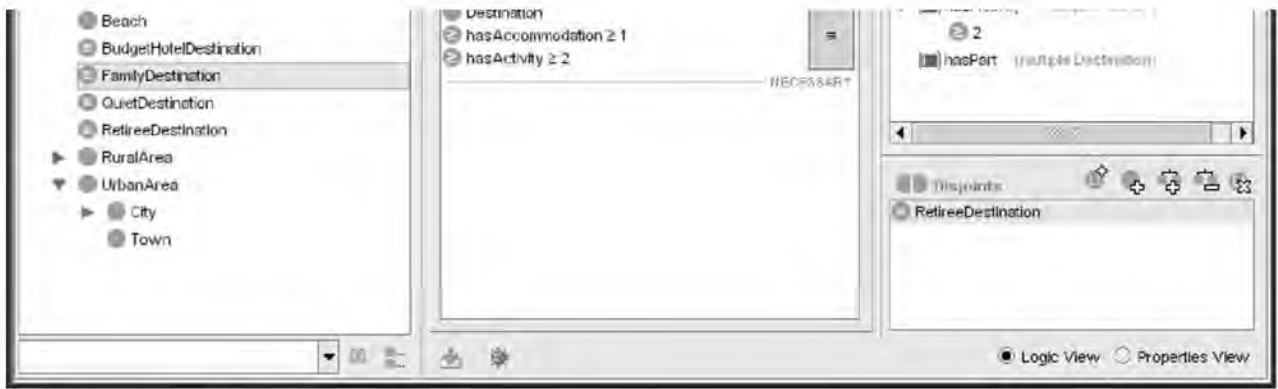
- *cardinality constraints* (e.g. a person has exactly one birth mother – no more and no less)
- *type constraints* (e.g. the age of a person would be a number – but not all numbers are legitimate values for ‘age’, and in particular, an age must be a *positive* number)
- *range constraints* (e.g. if the person class has a ‘mother’ property, which is supposed to identify the person’s mother, then the range is not just the ‘person’ class, since the ‘person’ class also includes men; the range would thus be the class ‘woman’)
- *domain constraints* (e.g. we might state that the property ‘author’ is not appropriate for the class ‘sculpture’ – the ‘domain’ here is the set of classes to which the property can be applied).

Step 7: Create instances

At this stage we populate the ontology with instances of classes. In general, we want to try to find the most specific class possible for all the objects that we want to identify and name, as this will convey the most information.

Figure 6.9: The Protégé ontology editor. Reproduced with permission from Stanford Center for Biomedical Informatics Research.





6.5 Software Tools for Ontologies

For the time being, most ontologies are crafted by human designers. To aid humans in the design of ontologies, many software tools have been developed that provide a user with, for example, graphical user interfaces to develop and display class hierarchies, and automated production of the code to deploy ontologies. One of the earliest and best-known such tools was the Ontolingua server [Farquhar et al., 1997]. The Ontolingua server is a web-based service that is intended to provide a common platform in which ontologies developed by different groups can be shared, and perhaps a common view of these ontologies achieved. The central component of Ontolingua is a library of ontologies, expressed in the Ontolingua ontology definition language (based on KIF). A server program provides access to this library. The library may be accessed through the server in several different ways: either by editing it directly (via a web-based interface), or by using programs that contact the server remotely via an NGFP interface. The Ontolingua server was capable of automatically transforming ontologies expressed in one format into a variety of others (e.g. the CORBA Interface Definition Language – IDL).

Probably the most widely used tool for ontology development is Protégé [Protégé Group, 2004]. Protégé is primarily a platform-independent ontology editor; it has a simple user interface for creating ontology class hierarchies, properties, and instances (see Figure 6.9) and can import and export such ontologies in a variety of formats, including OWL. However, Protégé also provides ‘plug-in’ support, which for example allows reasoning tools to be easily added to the basic system. Other ontology development environments with similar functionality include, for example, the NEON toolkit [NEON Project, 2008].

Notes and Further Reading

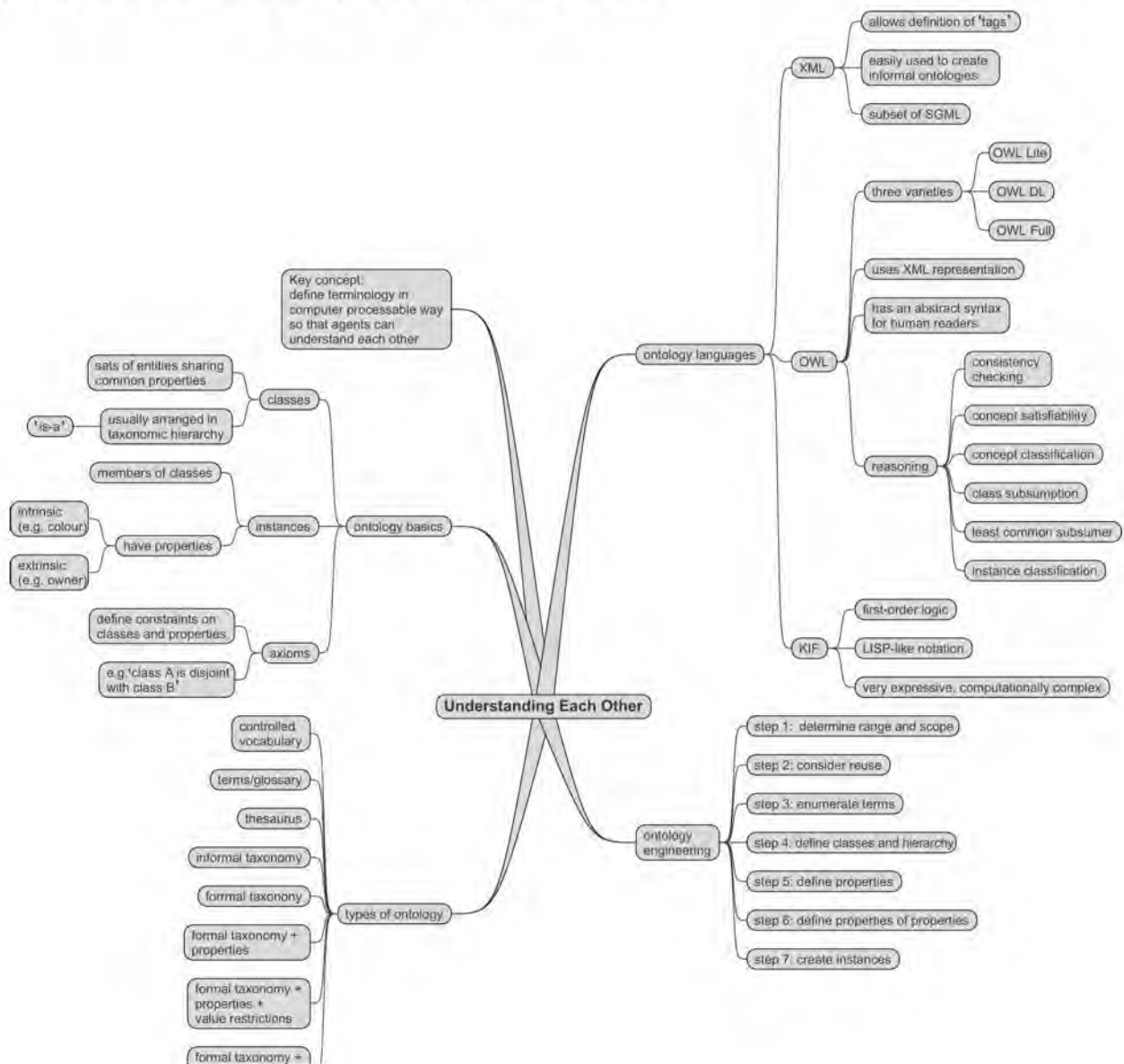
The literature on ontologies and their applications has exploded in recent years, and the literature is so big that it is difficult to know where to start in recommending further reading. I have to say that [Noy and McGuinness, 2004] is one of the most lucid introductions to ontologies that I have come across, although it is independent of any particular language and relatively informal. Most of the technical literature on ontologies is concerned with the relationship of ontology languages to description logics, and the associated reasoning problems. Probably the key reference to description logics is the handbook [Baader et al., 2003], which contains a comprehensive collection of (mostly quite technical) articles on the theory and practice of description logic, including reasoning problems. The *Handbook on Ontologies* provides another, more general collection of articles on ontologies, which includes, for example, descriptions of applications and methodologies for constructing ontologies [Staab and Studer, 2004]. A good general survey of work on ontologies up to 1996 is [Hendler and Glimberg, 1996]. A collection of KIF was published in [Glimberg, 1991].

1996 is [Uchold and Gruninger, 1996]. A critique of KIF was published as [Ginsberg, 1991]. Although we haven't covered reasoning with description logics (DLs), great progress has been made in recent years with algorithms for efficient reasoning with DLs, and a number of reasoning tools are available (see, for example, [Horrocks et al., 2000]).

The reason that ontologies have attracted so much attention recently is undoubtedly because of the semantic web. Since the original proposal of Berners-Lee [Berners-Lee, 1999; Berners-Lee et al., 2001], the vision has been elaborated and refined (and some would say hijacked) by many others. The introductory text [Antoniou and Harmelen, 2004] attempts to give a coherent introduction to the semantic web and the standards (such as OWL and RDF) that underpin it. [Singh and Huhns, 2005] gives a coherent and comprehensive overview of semantic web technologies and their role in multiagent systems. [Fensel et al., 2003] is an older collection of papers, which give a good feel for the origins of the semantic web. The main publication venue for current research in the semantic web is the *International Semantic Web Conference* (ISWC).

Class reading: introduction to [Berners-Lee et al., 2001]. This is the article that put some flesh on the bones of the term ‘semantic web’. It tries to set out what the key problems with the web are, and how ontologies might be used to overcome these. It is a popular science article, and doesn’t go into technical detail.

Figure 6.10: Mind map for this chapter.



properties +
arbitrary logical
constraints

Chapter 7 Communicating

Communication has long been recognized as a topic of central importance in computer science, and many formalisms have been developed for representing the properties of communicating concurrent systems [Hoare, 1978; Milner, 1989]. Such formalisms have tended to focus on a number of key issues that arise when dealing with systems that can interact with one another.

Perhaps the characteristic problem in communicating concurrent systems research is that of *synchronizing* multiple processes, which was widely studied throughout the 1970s and 1980s [Ben-Ari, 1990]. Essentially, two processes (cf. agents) need to be synchronized if there is a possibility that they can interfere with one another in a destructive way. The classic example of such interference is the ‘lost update’ scenario. In this scenario, we have two processes, p_1 and p_2 , both of which have access to some shared variable v . Process p_1 begins to update the value of v , by first reading it, then modifying it (perhaps by simply incrementing the value that it obtained), and finally saving this updated value in v . But between p_1 reading and again saving the value of v , process p_2 updates v , by saving some value in it. When p_1 saves its modified value of v , the update performed by p_2 is thus lost, which is almost certainly not what was intended. The lost update problem is a very real issue in the design of programs that communicate through shared data structures.

SYNCHRONIZATION

So, if we do not treat communication in such a ‘low-level’ way, then how *is* communication treated by the agent community? In order to understand the answer, it is helpful to first consider the way that communication is treated in the object-oriented programming community, that is, communication as method invocation. Suppose we have a Java system containing two objects, o_1 and o_2 , and that o_1 has a publicly available method m_1 . Object o_2 can communicate with o_1 by invoking method m_1 . In Java, this would mean o_2 executing an instruction that looks something like `o1.m1 (arg)`, where `arg` is the argument that o_2 wants to communicate to o_1 . But consider: which object makes the decision about the execution of method m_1 ? Is it object o_1 or object o_2 ? In this scenario, object o_1 has *no control* over the execution of m_1 : the decision about whether to execute m_1 lies entirely with o_2 .

Now consider a similar scenario, but in an agent-oriented setting. We have two agents i and j , where i has the capability to perform action α , which corresponds loosely to a method. But there is no concept in the agent-oriented world of agent j ‘invoking a method’ on i . This is because i is an *autonomous agent*: it has control over both its state and its behaviour. It cannot be taken for granted that agent i will execute action α just because another agent j wants it to. Performing the action α may not be in the best interests of agent i . The locus of control with respect to the decision about whether to execute an action is thus very different in agent and object systems.

In general, agents can neither force other agents to perform some action, nor write data onto the internal state of other agents. This does not mean that they cannot communicate, however. What they *can* do is perform actions – communicative actions – in an attempt to *influence* other agents appropriately. For example, suppose I say to you, ‘It is raining in London’, in a sincere way. Under normal circumstances, such a communication action is an attempt by me to modify your beliefs. Of course, simply uttering the sentence, ‘It is raining in London’, is not usually enough

to bring about this state of affairs, for all the reasons that were discussed above. You have control over your own beliefs (desires, intentions). You may believe that I am notoriously unreliable on the subject of the weather, or even that I am a pathological liar. But in performing the communication action of uttering, ‘It is raining in London’, I am attempting to change your internal state. Furthermore, since this utterance is an action that I perform, I am performing it for some purpose – presumably because I intend that you believe that it is raining.

7.1 Speech Acts

Speech act theory treats communication as action. It is predicated on the assumption that speech actions are performed by agents just like other actions, in the furtherance of their intentions.

SPEECH ACT THEORY

I begin with a historical overview of speech act theory, focusing in particular on attempts to develop formal theories of speech acts, where communications are modelled as actions that alter the mental state of communication participants.

7.1.1 Austin

The theory of speech acts is generally recognized to have begun with the work of the philosopher John Austin [Austin, 1962]. He noted that a certain class of natural language utterances – hereafter referred to as *speech acts* – had the characteristics of *actions*, in the sense that they change the state of the world in a way analogous to physical actions. It may seem strange to think of utterances changing the world in the way that physical actions do. If I pick up a block from a table (to use an overworked but traditional example), then the world has changed in an obvious way. But how does speech change the world? Austin gave as paradigm examples declaring war and saying ‘I now pronounce you man and wife’. Stated in the appropriate circumstances, these utterances clearly change the state of the world in a very tangible way.¹

Austin identified a number of *performative verbs*, which correspond to various different types of speech acts. Examples of such performative verbs are *request*, *inform*, and *promise*. In addition, Austin distinguished three different aspects of speech acts: the *locutionary act*, or act of making an utterance (e.g. saying ‘Please make some tea’), the *illocutionary act*, or action performed in saying something (e.g. ‘He requested me to make some tea’), and *perlocution*, or effect of the act (e.g. ‘He got me to make tea’).

PERFORMATIVES

LOCUTIONARY ACT

ILLOCUTION

PERLOCUTION

Austin referred to the conditions required for the successful completion of performatives as *felicity conditions*. He recognized three important felicity conditions.

FELICITY CONDITIONS

1. There must be an accepted conventional procedure for the performative, and the circumstances and persons must be as specified in the procedure.
2. The procedure must be executed correctly and completely.
3. The act must be sincere, and any *uptake* required must be completed, insofar as is possible.

7.1.2 Searle

Austin's work was extended by John Searle in his 1969 book *Speech Acts* [Searle, 1969]. Searle identified several properties that must hold for a speech act performed between a hearer and a speaker to succeed. For example, consider a *request* by SPEAKER to HEARER to perform ACTION.

Normal I/O conditions Normal I/O conditions state that HEARER is able to hear the request (thus must not be deaf, etc.); the act was performed in normal circumstances (not in a film or play, etc.); etc.

Preparatory conditions The preparatory conditions state what must be true of the world in order that SPEAKER correctly choose the speech act. In this case, HEARER must be able to perform ACTION, and SPEAKER must believe that HEARER is able to perform ACTION. Also, it must not be obvious that HEARER will do ACTION anyway.

Sincerity conditions These conditions distinguish sincere performances of the request; an insincere performance of the act might occur if SPEAKER did not really want ACTION to be performed.

Searle also attempted a systematic classification of possible types of speech acts, identifying the following five key classes.

Representatives A representative act commits the speaker to the truth of an expressed proposition. The paradigm case is *informing*.

REPRESENTATIVES

Directives A directive is an attempt on the part of the speaker to get the hearer to do something. Paradigm case: *requesting*.

DIRECTIVES

Commissives Commit the speaker to a course of action. Paradigm case: *promising*.

COMMISSIVES

Expressives Express some psychological state (gratitude, for example). Paradigm case: *thanking*.

EXPRESSIVES

Declarations Effect some changes in an institutional state of affairs. Paradigm case: *declaring war*.

DECLARATIONS

7.1.3 The plan-based theory of speech acts

In the late 1960s and early 1970s, a number of researchers in AI began to build systems that could plan how to autonomously achieve goals [Allen et al., 1990]. Clearly, if such a system is required to interact with humans or other autonomous agents, then such plans must include *speech* actions. This introduced the question of how the properties of speech acts could be represented such that planning systems could reason about them. [Cohen and Perrault, 1979] gave an account of the semantics of speech acts by using techniques developed in AI planning research [Fikes and Nilsson, 1971; Ghallab et al., 2004]. The aim of their work was to develop a theory of speech acts

... by modelling them in a planning system as operators defined ... in terms of speakers' and hearers' beliefs and goals. Thus speech acts are treated in the same way as physical actions.

[Cohen and Perrault, 1979]

The formalism chosen by Cohen and Perrault was the STRIPS notation, in which the properties of an action are characterized via preconditions and postconditions [Fikes and Nilsson, 1971]. The idea is very similar to Hoare logic [Hoare, 1969]. Cohen and Perrault demonstrated how the preconditions and postconditions of speech acts such as *request* could be represented in a multimodal logic containing operators for describing the *beliefs*, *abilities*, and *wants* of the participants in the speech act.

Consider the *Request* act. The aim of the *Request* act will be for a speaker to get a hearer to perform some action. [Figure 7.1](#) defines the *Request* act. Two preconditions are stated: the 'cando.pr' (can-do preconditions), and 'want.pr' (want preconditions). The cando.pr states that, for the successful completion of the *Request*, two conditions must hold. First, the speaker must believe that the hearer of the *Request* is able to perform the action. Second, the speaker must believe that the hearer also believes that it has the ability to perform the action. The want.pr states that, in order for the *Request* to be successful, the speaker must also believe it actually wants the *Request* to be performed. If the preconditions of the *Request* are fulfilled, then the *Request* will be successful: the result (defined by the 'effect' part of the definition) will be that the hearer believes that the speaker believes that it wants some action to be performed.

While the successful completion of the *Request* ensures that the hearer is aware of the speaker's desires, it is not enough in itself to guarantee that the desired action is actually performed. This is because the definition of *Request* only models the illocutionary force of the act. It says nothing of the perlocutionary force. What is required is a *mediating act*. [Figure 7.1](#) gives a definition of *CauseToWant*, which is an example of such an act.

MEDIATING ACT

Figure 7.1: Definitions from Cohen and Perrault's plan-based theory

or speech acts.

<i>Request</i> (<i>S</i> , <i>H</i> , α)		
Preconditions	Cando.pr	(<i>S</i> BELIEVE (<i>H</i> CANDO α)) $\wedge$ (<i>S</i> BELIEVE (<i>H</i> BELIEVE (<i>H</i> CANDO α)))
	Want.pr	(<i>S</i> BELIEVE (<i>S</i> WANT <i>requestInstance</i>))
Effect		(<i>H</i> BELIEVE (<i>S</i> BELIEVE (<i>S</i> WANT α)))
<i>CauseToWant</i> (<i>A</i> ₁ , <i>A</i> ₂ , α)		
Preconditions	Cando.pr	(<i>A</i> ₁ BELIEVE (<i>A</i> ₂ BELIEVE (<i>A</i> ₂ WANT α)))
	Want.pr	$\times$
Effect		(<i>A</i> ₁ BELIEVE (<i>A</i> ₁ WANT α))
<i>Inform</i> (<i>S</i> , <i>H</i> , φ)		
Preconditions	Cando.pr	(<i>S</i> BELIEVE φ)
	Want.pr	(<i>S</i> BELIEVE (<i>S</i> WANT <i>informInstance</i>))
Effect		(<i>H</i> BELIEVE (<i>S</i> BELIEVE φ))
<i>Convince</i> (<i>A</i> ₁ , <i>A</i> ₂ , φ)		
Preconditions	Cando.pr	(<i>A</i> ₁ BELIEVE (<i>A</i> ₂ BELIEVE φ))
	Want.pr	$\times$
Effect		(<i>A</i> ₁ BELIEVE φ)

By this definition, an agent will come to believe it wants to do something if it believes that another agent believes that it wants to do it. This definition could clearly be extended by adding more preconditions, perhaps to do with beliefs about social relationships, power structures, etc.

The *Inform* act is as basic as *Request*. The aim of performing an *Inform* will be for a speaker to get a hearer to believe some statement. Like *Request*, the definition of *Inform* requires an associated mediating act to model the perlocutionary force of the act. The cando.pr of *Inform* states that the speaker must believe φ is true. The effect of the act will simply be to make the hearer believe that the speaker believes φ . The cando.pr of *Convince* simply states that the hearer must believe that the speaker believes φ . The effect is simply to make the hearer believe φ .

7.1.4 Speech acts as rational action

While the plan-based theory of speech acts was a major step forward, it was recognized that a theory of speech acts should be rooted in a more general theory of rational action. This observation led Cohen and Levesque to develop a theory in which speech acts were modelled as actions performed by rational agents in the furtherance of their intentions [Cohen and Levesque, 1990b]. The foundation upon which they built this model of rational action was their theory of intention, described in [Cohen and Levesque, 1990a]. The formal theory is summarized in [Chapter 17](#), but, for now, here is the Cohen–Levesque definition of *requesting*, paraphrased in English.

⁴ A *request* is an attempt on the part of an agent to bring about a change in state of affairs.

A request is an attempt on the part of *spkr*, by doing *e*, to bring about a state where, ideally (i) *addr* intends α (relative to the *spkr* still having that goal, and *addr* still being helpfully inclined to *spkr*), and (ii) *addr* actually eventually does α , or at least brings about a state where *addr* believes it is mutually believed that it wants the ideal situation.

[Cohen and Levesque, [1990b](#), p. 241]

7.2 Agent Communication Languages

As I noted earlier, speech act theories have directly informed and influenced a number of languages that have been developed specifically for agent communication. In the early 1990s, the US-based DARPA-funded Knowledge Sharing Effort (KSE) was formed, with the remit of

[developing] protocols for the exchange of represented knowledge between autonomous information systems.

[Finin et al., [1993](#)]

The KSE generated two main deliverables as follows.

- The *knowledge query and manipulation language* (KQML). KQML is an ‘outer’ language for agent communication. It defines an ‘envelope’ format for messages, using which an agent can explicitly state the intended illocutionary force of a message. KQML is not concerned with the *content* part of messages [Mayfield et al., [1996](#); Patil et al., [1992](#)].

KQML

- The *knowledge interchange format* (KIF). KIF is a language explicitly intended to allow the representation of knowledge about some particular ‘domain of discourse’. It was intended primarily (though not uniquely) to form the content parts of KQML messages.

7.2.1 KQML

KQML is a message-based language for agent communication. Thus KQML defines a common format for messages. A KQML message may crudely be thought of as an object (in the sense of object-oriented programming): each message has a *performative* (which may be thought of as the class of the message), and a number of *parameters* (attribute/value pairs, which may be thought of as instance variables).

Table 7.1: Parameters for KQML messages.

Parameter	Meaning
:content	content of the message
:force	whether the sender of the message will ever deny the content of the message
:reply-with	whether the sender expects a reply, and, if so, an identifier for the reply
:in-reply-to	reference to the :reply-with parameter
:sender	sender of the message
:receiver	intended recipient of the message

Here is an example KQML message:

(ask-one


```
:content (PRICE IBM ?price)
:receiver stock-server
:language LPROLOG
:ontology NYSE-TICKS
)
```

The intuitive interpretation of this message is that the sender is asking about the price of IBM stock. The performative is `ask-one`, which an agent will use to ask a question of another agent where exactly one reply is needed. The various other components of this message represent its attributes. The most important of these is the `:content` field, which specifies the message content. In this case, the content simply asks for the price of IBM shares. The `:receiver` attribute specifies the intended recipient of the message, the `:language` attribute specifies that the language in which the content is expressed is called LPROLOG (the recipient is assumed to ‘understand’ LPROLOG), and the final `:ontology` attribute defines the *terminology* used in the message – the ontology may be defined using the techniques discussed in previous chapters. The main parameters used in KQML messages are summarized in [Table 7.1](#); note that different performatives require different sets of parameters.

Several different versions of KQML were proposed during the 1990s, with different collections of performatives in each. In [Table 7.2](#), I summarize the version of KQML performatives that appeared in [Finin et al., 1993]; this version contains a total of 41 performatives. In this table, *S* denotes the `:sender` of the messages, *R* denotes the `:receiver`, and *C* denotes the content of the message.

To more fully understand these performatives, it is necessary to understand the notion of a *virtual knowledge base* (VKB) as it was used in KQML. The idea was that agents using KQML to communicate may be implemented using different programming languages and paradigms – and, in particular, any information that agents have may be *internally* represented in many different ways. No agent can assume that another agent will use the same internal representation; indeed, no actual ‘representation’ may be present in an agent at all. Nevertheless, for the purposes of communication, it makes sense for agents to treat other agents *as if* they had some internal representation of knowledge. Thus agents *attribute* knowledge to other agents; this attributed knowledge is known as the virtual knowledge base.

[VIRTUAL KNOWLEDGE BASE](#)

Table 7.2: KQML performatives.

Performative	Meaning
achieve	<i>S</i> wants <i>R</i> to make something true of their environment
advertise	<i>S</i> claims to be suited to processing a performative
ask-about	<i>S</i> wants all relevant sentences in <i>R</i> ’s VKB
ask-all	<i>S</i> wants all of <i>R</i> ’s answers to a question <i>C</i>
ask-if	<i>S</i> wants to know whether the answer to <i>C</i> is in <i>R</i> ’s VKB
ask-one	<i>S</i> wants one of <i>R</i> ’s answers to question <i>C</i>
break	<i>S</i> wants <i>R</i> to break an established pipe
broadcast	<i>S</i> wants <i>R</i> to send a performative over all connections
broker-all	<i>S</i> wants <i>R</i> to collect all responses to a performative
broker-one	<i>S</i> wants <i>R</i> to get help in responding to a performative

deny	the embedded performative does not apply to <i>S</i> (any more)
delete-all	<i>S</i> wants <i>R</i> to remove all sentences matching <i>C</i> from its VKB
delete-one	<i>S</i> wants <i>R</i> to remove one sentence matching <i>C</i> from its VKB
discard	<i>S</i> will not want <i>R</i> 's remaining responses to a query
eos	end of a stream response to an earlier query
error	<i>S</i> considers <i>R</i> 's earlier message to be malformed
evaluate	<i>S</i> wants <i>R</i> to evaluate (simplify) <i>C</i>
forward	<i>S</i> wants <i>R</i> to forward a message to another agent
generator	same as a <code>standby</code> of a <code>stream-all</code>
insert	<i>S</i> asks <i>R</i> to add content to its VKB
monitor	<i>S</i> wants updates to <i>R</i> 's response to a <code>stream-all</code>
next	<i>S</i> wants <i>R</i> 's next response to a previously streamed performative
pipe	<i>S</i> wants <i>R</i> to route all further performatives to another agent
ready	<i>S</i> is ready to respond to <i>R</i> 's previously mentioned performative
recommend-all	<i>S</i> wants all names of agents who can respond to <i>C</i>
recommend-one	<i>S</i> wants the name of an agent who can respond to a <i>C</i>
recruit-all	<i>S</i> wants <i>R</i> to get all suitable agents to respond to <i>C</i>
recruit-one	<i>S</i> wants <i>R</i> to get one suitable agent to respond to <i>C</i>
register	<i>S</i> can deliver performatives to some named agent
reply	communicates an expected reply
rest	<i>S</i> wants <i>R</i> 's remaining responses to a previously named performative
sorry	<i>S</i> cannot provide a more informative reply
standby	<i>S</i> wants <i>R</i> to be ready to respond to a performative
stream-about	multiple response version of <code>ask-about</code>
stream-all	multiple response version of <code>ask-all</code>
subscribe	<i>S</i> wants updates to <i>R</i> 's response to a performative
tell	<i>S</i> claims to <i>R</i> that <i>C</i> is in <i>S</i> 's VKB
transport-address	<i>S</i> associates symbolic name with transport address
unregister	the deny of a <code>register</code>
untell	<i>S</i> claims to <i>R</i> that <i>C</i> is <i>not</i> in <i>S</i> 's VKB

Figure 7.2: Example KQML dialogues.

Dialogue (a)

```
(evaluate
  :sender A :receiver B
  :language KIF :ontology motors
  :reply-with q1 :content (val (torque m1)))

(reply
  :sender B :receiver A
  :language KIF :ontology motors
  :in-reply-to q1 :content (= (torque m1) (scalar 12 kgf.m)))
```

Dialogue (b)

```
(stream-about
  :sender A :receiver B
  :language KIF :ontology motors
  :reply-with q1 :content m1)

(tell
```



```

:sender B :receiver A
:in-reply-to q1 :content (= (torque m1) (scalar 12 kgf.m)))
(tell
:sender B :receiver A
:in-reply-to q1 :content (= (status m1) normal))
(eos
:sender B :receiver A
:in-reply-to q1)

```

Example KQML dialogues

To illustrate the use of KQML, we will now consider some example KQML dialogues (these examples are adapted from [Finin et al., 1993]). In the first dialogue ([Figure 7.2\(a\)](#)), agent A sends to agent B a query, and subsequently gets a response to this query. The query is the value of the torque on *m1*; agent A gives the query the name *q1* so that B can later refer back to this query when it responds. Finally, the `:ontology` of the query is `motors` – as might be guessed, this ontology defines a terminology relating to motors. The response that B sends indicates that the torque of *m1* is equal to 12 kgf.m – a scalar value.

The second dialogue ([Figure 7.2\(b\)](#)) illustrates a *stream* of messages: agent A asks agent B for everything it knows about *m1*. Agent B responds with two `tell` messages, indicating what it knows about *m1*, and then sends an `eos` (end of stream) message, indicating that it will send no more messages about *m1*. The first `tell` message indicates that the torque of *m1* is 12kgf.m (as in dialogue (a)); the second `tell` message indicates that the status of *m1* is normal. Note that there is no content to the `eos` message; `eos` is thus a kind of meta-message – a message about messages.

The third (and most complex) dialogue, shown in [Figure 7.3](#), shows how KQML messages themselves can be the content of KQML messages. The dialogue begins when agent A advertises to agent B that it is willing to accept subscriptions relating to *m1*. Agent B responds by subscribing to agent A with respect to *m1*. Agent A then responds with sequence of messages about *m1*; as well as including `tell` messages, as we have already seen, the sequence includes an `untell` message, to the effect that the torque of *m1* is no longer 12 kgf.m, followed by a `tell` message indicating the new value of the torque. The sequence ends with an end of stream message.

The take-up of KQML by the multiagent systems community was significant, and several KQML-based implementations were developed and distributed. Despite this success, KQML was subsequently criticized on a number of grounds as follows.

- The basic KQML performative set was rather fluid – it was never tightly constrained, and so different implementations of KQML were developed that could not, in fact, interoperate.
- Transport mechanisms for KQML messages (i.e. ways of getting a message from agent A to agent B) were never precisely defined, again making it hard for different KQML-talking agents to interoperate.
- The semantics of KQML was never rigorously defined, in such a way that it was possible to tell whether two agents claiming to be talking KQML were in fact using the language ‘properly’. The ‘meaning’ of KQML performatives was only defined using informal, English language descriptions, open to different interpretations. (I discuss

this issue in more detail later on in this chapter.)

- The language was missing an entire class of performatives – *commissives*, by which one agent makes a commitment to another. As Cohen and Levesque point out, it is difficult to see how many multiagent scenarios could be implemented without commissives, which appear to be important if agents are to *coordinate* their actions with one another.
- The performative set for KQML was overly large and, it could be argued, rather ad hoc.

These criticisms – among others – led to the development of a new, but rather closely related, language by the FIPA consortium.

7.2.2 The FIPA agent communication language

In 1995, the Foundation for Intelligent Physical Agents (FIPA) began its work on developing standards for agent systems. The centerpiece of this initiative was the development of an agent communication language (ACL) [FIPA, 1999]. This ACL is superficially similar to KQML: it defines an ‘outer’ language for messages, it defines 20 performatives (such as `inform`) for defining the intended interpretation of messages, and it does not mandate any specific language for message content. In addition, the concrete syntax for FIPA ACL messages closely resembles that of KQML. Here is an example of a FIPA ACL message (from [FIPA, 1999]):

FIPA

Figure 7.3: Another KQML dialogue.

Dialogue (c)

```
(advertise
:sender A
:language KQML :ontology K10
:content
  (subscribe
    :language KQML :ontology K10
    :content
      (stream-about
        :language KIF :ontology motors
        :content m1)))
(subscribe
:sender B :receiver A
:reply-with s1
:content
  (stream-about
    :language KIF :ontology motors
    :content m1))
(tell
:sender A :receiver B
:in-reply-to s1 :content (= (torque m1) (scalar 12 kgf.m)))
(tell
:sender A :receiver B
:in-reply-to s1 :content (= (status m1) normal))
(untell
:sender A :receiver B
```



```

:sender A :receiver B
:in-reply-to s1 :content (= (torque m1) (scalar 12 kgf.m)))
(tell
:sender A :receiver B
:in-reply-to s1 :content (= (torque m1) (scalar 15 kgf.m)))
(eos
:sender A :receiver B
:in-reply-to s1)

(inform
:sender agent1

```

Table 7.3: Performatives provided by the FIPA communication language.

Performative	Passing information	Requesting information	Negotiation	Performing actions	Error handling
accept-proposal			x		
agree				x	
cancel		x		x	
cfp			x		
confirm	x				
disconfirm	x				
failure					x
inform	x				
inform-if	x				
inform-ref	x				
not-understood					x
propagate				x	
propose			x		
proxy				x	
query-if		x			
query-ref		x			
refuse				x	
reject-proposal			x		
request				x	
request-when				x	
request-whenever				x	
subscribe		x			

```

:receiver agent2
:content (price good2 150)
:language s1
:ontology hpl-auction
)

```

As should be clear from this example, the FIPA communication language is similar to KQML: the structure of messages is the same, and the message attribute fields are also very similar. The relationship between the FIPA ACL and KQML is discussed in [FIPA, 1999, pp. 68–69]. The most important difference between the two languages is in the collection of

so on). The most important difference between the two languages is in the collection of performatives they provide. The performatives provided by the FIPA communication language are categorized in [Table 7.3](#).

Informally, these performatives have the following meaning.

`accept-proposal` – The `accept-proposal` performative allows an agent to state that it accepts a `proposal` made by another agent.

`agree` – An `agree` performative is used by one agent to indicate that it has acquiesced to a request made by another agent. It indicates that the sender of the `agree` message intends to carry out the requested action.

`cancel` – A `cancel` performative is used by an agent to follow up a previous request message, and indicates that it no longer desires a particular action to be carried out.

`cfp` – A `cfp` (call for proposals) performative is used to initiate negotiation between agents. The content attribute of a `cfp` message contains both an action (e.g. ‘sell me a car’) and a condition (e.g. ‘the price of the car is less than US\$10 000’). Essentially, it says ‘here is an action that I wish to be carried out, and here are the terms under which I want it to be carried out – send me your proposals’. (We will see in the next chapter that the `cfp` message is a central component of *task-sharing* systems such as the *Contract Net*.)

`confirm` – The `confirm` performative allows the sender of the message to confirm the truth of the content to the recipient, where, before sending the message, the sender believes that the recipient is unsure about the truth or otherwise of the content.

`disconfirm` – Similar to `confirm`, but this performative indicates to a recipient that is unsure as to whether or not the sender believes the content that the content is in fact false.

`failure` – This allows an agent to indicate to another agent that an attempt to perform some action (typically, one that it was previously requested to perform) failed.

`inform` – Along with `request`, the `inform` performative is one of the two most important performatives in the FIPA ACL. It is the basic mechanism for communicating information. The content of an `inform` performative is a statement, and the idea is that the sender of the `inform` wants the recipient to believe this content. Intuitively, the sender is also implicitly stating that *it* believes the content of the message.

`inform-if` – An `inform-if` implicitly says either that a particular statement is true or that it is false. Typically, an `inform-if` performative forms the content part of a message. An agent will send a `request` message to another agent, with the content part being an `inform-if` message. The idea is that the sender of the `request` is saying ‘tell me if the content of the `inform-if` is either true or false’.

`inform-ref` – The idea of `inform-ref` is somewhat similar to that of `inform-if`; the difference is that, rather than asking whether or not an expression is true or false, the agent asks for the *value* of an expression.

`not-understood` – This performative is used by one agent to indicate to another agent that it recognized that it performed some action, but did not understand why this action was performed. The most common use of `not-understood` is for one agent to indicate to another agent that a message that was just received was not understood. The content part of a `not-understood` message consists of both an action (the one whose purpose was not understood) and a statement, which gives some explanation of why it was not understood.

This performative is the central error-handling mechanism in the FIPA ACL.

`propagate` – The content attribute of a `propagate` message consists of two things: another message, and an expression that denotes a set of agents. The idea is that the recipient of the `propagate` message should send the embedded message to the agent(s) denoted by this expression.

`propose` – This performative allows an agent to make a proposal to another agent, for example in response to a `cfp` message that was previously sent out.

`proxy` – The `proxy` message type allows the sender of the message to treat the recipient of the message as a `proxy` for a set of agents. The content of a `proxy` message will contain both an embedded message (one that it wants forwarded to others) and a specification of the agents that it wants the message forwarded to.

`query-if` – This performative allows one agent to ask another whether or not some specific statement is true. The content of the message will be the statement that the sender wishes to enquire about.

`query-ref` – This performative is used by one agent to determine a specific value for an expression (cf. the `evaluate` performative in KQML).

`refuse` – A `refuse` performative is used by one agent to state to another agent that it will not perform some action. The message content will contain both the action and a sentence that characterizes why the agent will not perform the action.

`reject-proposal` – Allows an agent to indicate to another that it does not accept a `proposal` that was made as part of a negotiation process. The content specifies both the `proposal` that is being rejected, and a statement that characterizes the reasons for this rejection.

`request` – The second fundamental performative allows an agent to request another agent to perform some action.

`request-when` – The content of a `request-when` message will be both an action and a statement; the idea is that the sender wants the recipient to carry out the action when the statement is true (e.g. ‘sound the bell when the temperature falls below 20 Celsius’).

`request-whenever` – Similar to `request-when`, but the idea is that the recipient should perform the action *whenever* the statement is true.

`subscribe` – Essentially as in KQML: the content will be a statement, and the sender wants to be notified whenever something relating to the statement changes.

Semantics of the FIPA ACL

Given that one of the most frequent and damning criticisms of KQML was the lack of an adequate semantics, it is perhaps not surprising that the developers of the FIPA agent communication language felt it important to give a comprehensive formal semantics to their language. The approach adopted drew heavily on Cohen and Levesque’s theory of speech acts as rational action [Cohen and Levesque, 1990b], but in particular on Sadek’s enhancements to this work [Bretier and Sadek, 1997]. The semantics was given with respect to a formal language called SL. This language allows one to represent *beliefs*, *desires*, and *uncertain beliefs* of agents, as well as the actions that agents perform. The semantics of the FIPA ACL maps each ACL message to a formula of SL that defines a constraint that the sender of the message must satisfy if it is to be considered as conforming to the FIPA ACL

sender of the message must satisfy if it is to be considered as conforming to the FIPA ACL standard. FIPA refers to this constraint as the *feasibility* condition. The semantics also maps each message to an SL-formula that defines the *rational effect* of the action – the ‘purpose’ of the message: what an agent will be attempting to achieve in sending the message (cf. perlocutionary act). However, in a society of autonomous agents, the rational effect of a message cannot (and should not) be guaranteed. Hence conformance does not require the recipient of a message to respect the rational effect part of the ACL semantics – only the feasibility condition.

FIPA SEMANTICS

RATIONAL EFFECT

As I noted above, the two most important communication primitives in the FIPA languages are `inform` and `request`. In fact, *all* other performatives in FIPA are defined in terms of these performatives. Here is the semantics for `inform` [FIPA, 1999, p. 25]:

$$(i, \text{inform}(j, \phi)) \quad (7.1)$$

$$\text{feasibility precondition: } B_i \phi \vee \neg B_i (B \text{ if}_j \phi \vee u \text{ if}_j \phi) \quad (7.2)$$

$$\text{rational effect: } B_j \phi. \quad (7.3)$$

The $B_i \phi$ means ‘agent i believes ϕ ’; $B \text{ if}_i \phi$ means that ‘agent i has a definite opinion one way or the other about the truth or falsity of ϕ ’; and $U \text{ if}_i \phi$ means that agent i is ‘uncertain’ about ϕ . Thus an agent i sending an `inform` message with content ϕ to agent j will be respecting the semantics of the FIPA ACL if it believes ϕ , and it is not the case that it believes of j either that j believes whether ϕ is true or false, or that j is uncertain of the truth or falsity of ϕ . If the agent is *successful* in performing the `inform`, then the recipient of the message – agent j – will believe ϕ .

The semantics of `request` is as follows:²

$$(i, \text{request}(j, \alpha)) \quad (7.4)$$

$$\text{feasibility precondition: } B_i \text{Agent}(\alpha, j) \vee \neg B_i I_j \text{Done}(\alpha) \quad (7.5)$$

$$\text{rational effect: } \text{Done}(\alpha). \quad (7.6)$$

The SL expression $\text{Agent}(\alpha, j)$ means that the agent of action α is j (i.e. j is the agent who performs α); and $\text{Done}(\alpha)$ means that the action α has been done. Thus agent i requesting agent j to perform action α means that agent i believes that the agent of α is j (and so it is sending the message to the right agent), and agent i believes that agent j does not currently intend that α is done. The rational effect – what i wants to achieve by this – is that the action is done.

Conformance testing

One key issue for this work is that of *semantic conformance testing*. The conformance

testing problem can be summarized as follows [Wooldridge, [1998](#)]. We are given an agent, and an agent communication language with a well-defined semantics. The aim is to determine whether or not the agent respects the semantics of the language whenever it communicates. *Syntactic* conformance testing is of course easy – the difficult part is to see whether or not a particular agent program respects the *semantics* of the language.

SEMANTIC CONFORMANCE TESTING

The importance of conformance testing *has* been recognized by the FIPA ACL community [FIPA, [1999](#), p. 1]. However, to date, little research has been carried out either on how verifiable communication languages might be developed, or on how existing ACLs might be verified. One exception is (my) [Wooldridge, [1998](#)], where the issue of conformance testing is discussed from a formal point of view: I point out that the ACL semantics is generally developed in such a way as to express *constraints* on the senders of messages. For example, the constraint imposed by the semantics of an ‘inform’ message might state that the sender believes the message content. This constraint can be viewed as a *specification*. Verifying that an agent respects the semantics of the agent communication language then reduces to a conventional program verification problem: show that the agent sending the message satisfies the specification given by the communication language semantics. But to solve this verification problem, we would have to be able to talk about the mental states of agents – what they believed, intended and so on. Given an agent implemented in (say) Java, it is not clear how this might be done.

7.2.3 JADE

To facilitate the rapid development of multiagent systems using the FIPA ACL, several platforms have been developed that support FIPA messaging. Of these, the best-known and most widely used is the *Java Agent Development Environment* (JADE) [Bellifemine et al., [2007](#)]. Initially developed by Telecom Italia in 1998, JADE intends to provide software packages and tools to allow Java developers to deploy FIPA agent systems.

Agents in JADE are implemented as Java threads (lightweight processes). JADE agents ‘live’ in a *container*, which is the Java process that launches them. At runtime, a JADE system will contain multiple containers, of which one will be designated as the *main container*. The main container is the ‘bootstrap’ container, and all containers in a system must register with the main container. The main container has the following functions [Bellifemine et al., [2007](#), pp. 32–33]:

JADE CONTAINER

MAIN CONTAINER

- maintain a *container table*, which lists all containers in the system and how to contact them
- maintain a *global agent descriptor table*, which lists all agents in the system, including current status and location
- host the *agent management system*, which provides services for naming agents using *agent identifiers*, as well as creating and destroying agents

Figure 7.4: Sending and receiving FIPA messages with JADE (line numbers are for reference only, and are not part of the code).

```
1.    // create & send CFP message to several recipients
2.    ACLMessage cfp = new ACLMessage(ACLMessage.CFP);
3.    for (int i = 0; i < numOfRecipients; ++i) {
4.        cfp.addReceiver(sellerAgents[i]);
5.    }
6.    cfp.setContent(targetBookTitle);
7.    cfp.setConversationId("book-trade");
8.    cfp.setReplyWith("cfp"+System.currentTimeMillis());
9.    myAgent.send(cfp);
10.   [...]
11.   // Prepare the template to receive proposals
12.   mt = MessageTemplate.and(
13.       MessageTemplate.MatchConversationId("book-trade"),
14.       MessageTemplate.MatchInReplyTo(cfp.getReplyWith()));
15.   // Receive all proposals/refusals from seller agents
16.   ACLMessage reply = myAgent.receive(mt);
17.   if (reply != null) {
18.       // Reply received
19.       if (reply.getPerformative() == ACLMessage.PROPOSE) {
20.           // This is an offer
21.           int price = Integer.parseInt(reply.getContent());
22.           if (bestSeller == null || price < bestPrice) {
23.               // This is the best offer at present
24.               bestPrice = price;
25.               bestSeller = reply.getSender();
26.           }
27.       }
```

- host the *directory facilitator*, which provides a ‘yellow pages’ service, allowing agents to register their services, and identify agents by the services they provide.

One important principle in JADE is that communication is transparent, in the sense that the programmer need not be aware of the mechanism used to actually deliver messages.

Communication is handled by the JADE run-time environment, which also provides a number of GUI-based tools to enable a user to monitor and control run-time system activity.

[Figure 7.4](#) gives an example fragment of Java code using JADE. The idea of the code is that an agent is trying to find the best deal possible for buying a book. It sends out a `cfp` message to a number of recipients (listed in the array `seller`), and then awaits responses, identifying the best price from the replies (the agent offering the lowest price is held in the variable `bestSeller`).

Lines 2–8 involve the creation of the message to be sent: `ACLMessage` is the Java class for FIPA messages, and the constructor for this class takes a single parameter, the type of the message to be created (in this case, `CFP`, which is defined as a static variable in the `ACLMessage` class). Lines 6–8 set the parameters of the message. Line 9 causes the message to be sent.

Lines 12–27 handle the receipt and processing of replies. First, in lines 12–14, we create a *message template* which allows us to filter the messages we are looking for (i.e. replies to our

message content, then use it to filter the messages we are looking for (in reply to our CFP message). Line 16 gets a reply. In line 19, we check that the message is a PROPOSE, i.e. the sender of the response is making a proposal with respect to our request. If so, then in line 21 we extract the proposed price from the content of the message (assumed to be an integer), and then in lines 22–26, we check to see whether this is the best proposal yet received, and if so, keep track of the proposer in the `bestSeller` variable.

Notes and Further Reading

The problems associated with communicating concurrent systems have driven a significant fraction of research into theoretical computer science since the early 1980s. Two of the best-known formalisms developed in this period are Tony Hoare's Communicating Sequential Processes (CSPs) [Hoare, [1978](#)], and Robin Milner's Calculus of Communicating Systems (CCS) [Milner, [1989](#)]. Temporal logic has also been widely used for reasoning about concurrent systems – see, for example, [Pnueli, [1986](#)] for an overview. A good reference that describes the key problems in concurrent and distributed systems is [Ben-Ari, [1990](#)].

The plan-based theory of speech acts developed by Cohen and Perrault made speech act theory accessible and directly usable to the artificial intelligence community [Cohen and Perrault, [1979](#)]. In the multiagent systems community, this work is arguably the most influential single publication on the topic of speech-act-like communication. Many authors have built on its basic ideas. For example, borrowing a formalism for representing the mental state of agents that was developed by Robert Moore [Moore, [1990](#)], Douglas Appelt was able to implement a system that was capable of planning to perform speech acts [Appelt, [1982](#), [1985](#)].

Many other approaches to speech act semantics have appeared in the literature. For example, Perrault [Perrault, [1990](#)] described how Reiter's default logic [Reiter, [1980](#)] could be used to reason about speech acts. Singh developed a theory of speech acts [Singh, [1991c](#), [1993](#)] using his formal framework for representing rational agents [Singh, [1990a,b](#), [1991a,b](#), [1994](#), [1998b](#); Singh and Asher, [1991](#)]. He introduced a predicate $comm(i, j, m)$ to represent the fact that agent i communicates message m to agent j , and then used this predicate to define the semantics of assertive, directive, commissive, and permissive speech acts.

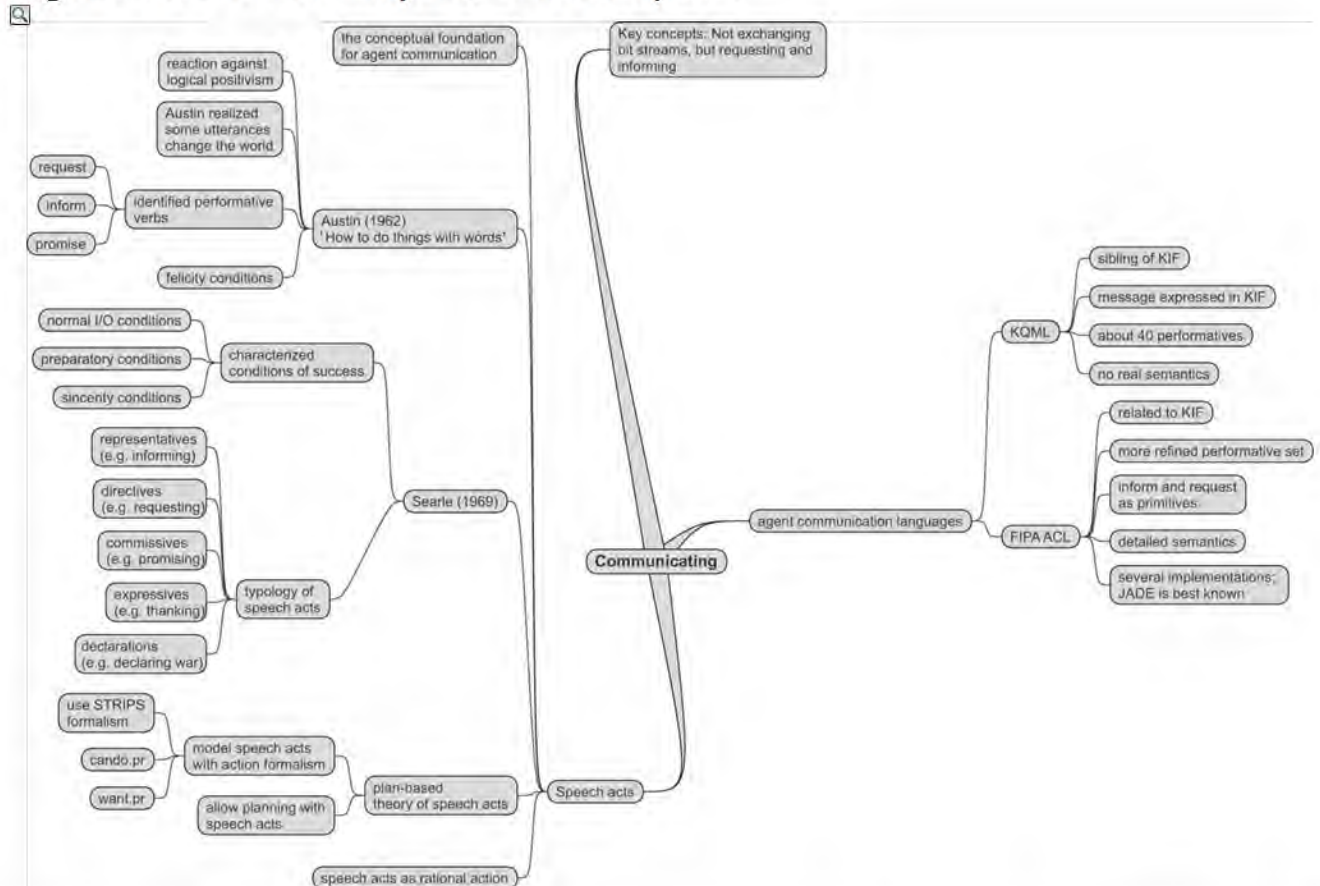
[Dignum and Greaves, [2000](#)] is a collection of papers on agent communication languages. As I mentioned in the main text of the chapter, a number of KQML implementations have been developed: well-known examples are InfoSleuth [Nodine and Unruh, [1998](#)], KAoS [Bradshaw et al., [1997](#)] and JATLite [Jeon et al., [2000](#)]. The definitive reference to JADE is [Bellifemine et al., [2007](#)].

Recently, a number of proposals have appeared for communication languages with a verifiable semantics [Pitt and Mamdani, [1999](#); Singh, [1998a](#); Wooldridge, [1999](#)]. See [Labrou et al., [1999](#)] for a discussion of the state of the art in agent communication languages as of early 1999.

Coordination languages have been the subject of much interest by the theoretical computer science community; a regular conference is now held on the subject, the proceedings of which were published as [Ciancarini and Hankin, [1996](#)]. The Linda coordination model has been implemented in the JavaSpaces package [Freeman et al., [1999](#)], making it possible to

Class discussion: [Singh, 1998a] This is an overview of the state of the art in agent communication (as at 1998), and an introduction to the key challenges, particularly with respect to the semantics of agent communication.

Figure 7.5: Mind map for this chapter.



¹Notice that when referring to the effects of communication, I am ignoring 'pathological' cases, such as shouting while on a ski run and causing an avalanche. Similarly, I will ignore 'microscopic' effects (such as the minute changes in pressure or temperature in a room caused by speaking).

²In the interests of comprehension, I have simplified the semantics a little.

Chapter 8 Working Together

So far, we have seen nothing of how agents can *work together*. In this chapter, we rectify this. We will see how agents can be designed so that they can work together effectively. As I noted in [Chapter 1](#), the idea of computer systems working together may not initially appear to be very novel: the term ‘cooperation’ is frequently used in the concurrent systems literature to describe systems that must interact with one another in order to carry out their assigned tasks. There are two main distinctions between multiagent systems and ‘traditional’ distributed systems:

- Agents in a multiagent system may have been designed and implemented by different individuals, with different goals. They therefore might not share common goals, and so, as we saw in earlier chapters, the encounters between agents in a multiagent system more closely resemble *games*, in which agents must act strategically in order to achieve the outcome they most prefer.
- Because agents are assumed to be acting autonomously (and so making decisions about what to do *at run-time*, rather than having all decisions hardwired in at design time), they must be capable of *dynamically* coordinating their activities and cooperating with others. In traditional distributed and concurrent systems, coordination and cooperation are typically hardwired in at design time.

Working together involves several different kinds of activities, which we will investigate in much more detail throughout this chapter: in particular, the sharing both of tasks and of information, and the dynamic (i.e. run-time) coordination of multiagent activities.

8.1 Cooperative Distributed Problem Solving

Work on *cooperative distributed problem solving* (CDPS) began with the work of Lesser and colleagues on systems that contained agent-like entities, each of which had distinct (but interrelated) expertise that they could bring to bear on problems that the entire system is required to solve:

CDPS

CDPS studies how a loosely coupled network of problem solvers can work together to solve problems that are beyond their individual capabilities. Each problem solving node in the network is capable of sophisticated problem solving and can work independently, but the problems faced by the nodes cannot be completed without cooperation. Cooperation is necessary because no single node has sufficient expertise, resources, and information to solve a problem, and different nodes might have expertise for solving different parts of the problem.

[Durfee et al., [1989b](#), p. 63]

Historically, most work on cooperative problem solving has made the *benevolence* assumption: that the agents in a system implicitly share a common goal, and thus that there is no potential for conflict between them. This assumption implies that agents can be designed so as to help out whenever needed, even if it means that one or more agents must suffer in order to do so. Intuitively, all that matters is the *overall* system objectives, not those of the individual agents within it. The benevolence assumption is generally acceptable if all the agents in a system are designed or ‘owned’ by the same organization or individual. It is important to emphasize that

the ability to assume benevolence *greatly* simplifies the designer's task. If we can assume that all that the agents need to worry about is the overall utility of the system, then we can design the overall system so as to optimize this.

BENEVOLENCE ASSUMPTION

In contrast to work on distributed problem solving, the more general area of multiagent systems has focused on the issues associated with societies of *self-interested* agents. Thus, agents in a multiagent system (unlike those in typical distributed problem-solving systems) cannot be assumed to share a common goal, as they will often be designed by different individuals or organizations in order to represent their interests. One agent's interests may therefore conflict with those of others, just as in human societies. Despite the potential for conflicts of interest, the agents in a multiagent system will ultimately need to cooperate in order to achieve their goals; again, just as they do in human societies. Multiagent systems research is therefore concerned with the wider problems of designing societies of autonomous agents, such as why and how agents cooperate [Wooldridge and Jennings, 1994]; how agents can recognize and resolve conflicts [Adler et al., 1989; Galliers, 1988b; Lander et al., 1991]; how agents can negotiate or compromise in situations where they are apparently at loggerheads [Ephrati and Rosenschein, 1993; Rosenschein and Zlotkin, 1994]; and so on.

It is also important to distinguish CDPS from *parallel problem solving* [Bond and Gasser, 1988, p. 3]. Parallel problem solving simply involves the exploitation of parallelism in solving problems. Typically, in parallel problem solving, the computational components are simply processors; a single node will be responsible for *decomposing* the overall problem into subcomponents, allocating these to processors, and subsequently assembling the solution. The nodes are frequently assumed to be homogeneous in the sense that they do not have distinct expertise – they are simply processors to be exploited in solving the problem. Although parallel problem solving was synonymous with CDPS in the early days of multiagent systems, the two fields are now regarded as quite separate. (However, it goes without saying that a multiagent system will employ parallel architectures and languages: the point is that the concerns of the two areas are rather different.)

PARALLEL PROBLEM SOLVING

Coherence and coordination

Having implemented an artificial agent society in order to solve some problem, how does one assess the success (or otherwise) of the implementation? What criteria can be used? The multiagent systems literature has proposed two types of issues that need to be considered.

Coherence refers to 'how well the [multiagent] system behaves as a unit, along some dimension of evaluation' [Bond and Gasser, 1988, p.19]. Coherence may be measured in terms of solution quality, efficiency of resource usage, conceptual clarity of operation, or how well system performance degrades in the presence of uncertainty or failure; a discussion on the subject of when multiple agents can be said to be acting coherently appears as [Wooldridge, 1994].

Coordination, in contrast, is 'the degree ... to which [the agents] ... can avoid "extraneous" activity [such as] ... synchronizing and aligning their activities' [Bond and

Gasser, 1988, p. 19]; in a perfectly coordinated system, agents will not accidentally clobber each other's subgoals while attempting to achieve a common goal; they will not need to explicitly communicate, as they will be mutually predictable, perhaps by maintaining good internal models of each other. The presence of conflict between agents, in the sense of agents destructively interfering with one another (which requires time and effort to resolve), is an indicator of poor coordination.

It is probably true to say that these problems have been the focus of more attention in multiagent systems research than any other issues [Durfee, 1988; Durfee and Lesser, 1987; Gasser and Hill Jr., 1990; Goldman and Rosenschein, 1994; Jennings, 1993a; Weiß, 1993].

The main issues to be addressed in CDPS include the following.

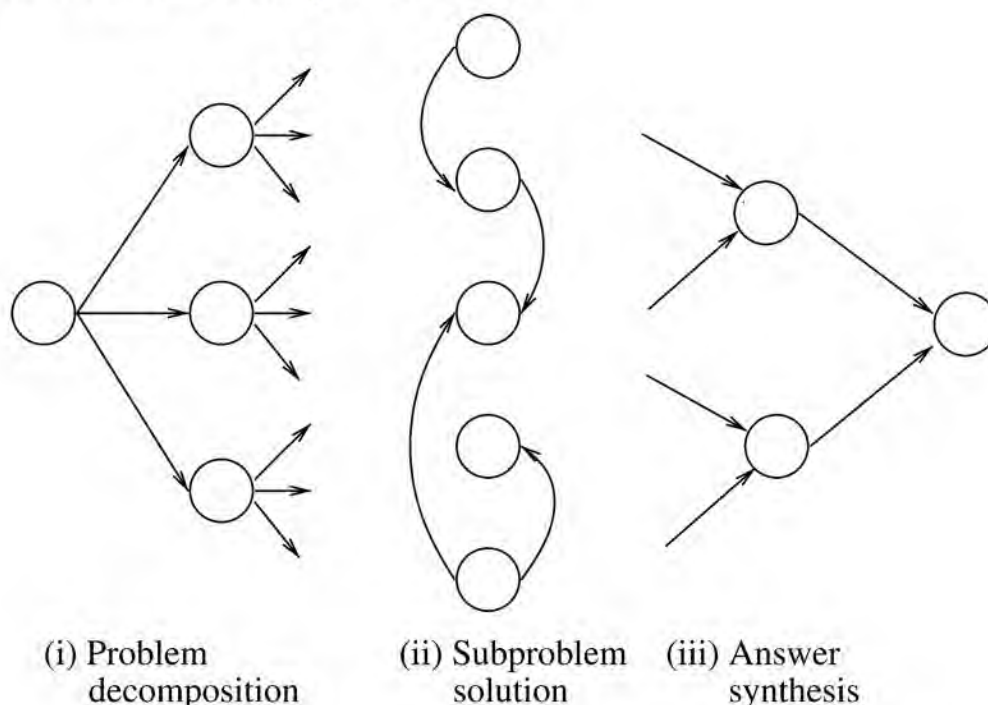
- How can a problem be divided into smaller tasks for distribution among agents?
- How can a problem solution be effectively synthesized from subproblem results?
- How can the overall problem-solving activities of the agents be optimized so as to produce a solution that maximizes the coherence metric?
- What techniques can be used to coordinate the activity of the agents, thus avoiding destructive (and therefore unhelpful) interactions, and maximizing effectiveness (by exploiting any positive interactions)?

In the remainder of this chapter, we shall see some techniques developed by the multiagent systems community for addressing these concerns.

8.2 Task Sharing and Result Sharing

How does a group of agents work together to solve problems? [Smith and Davis, 1980] suggested that the CDPS process can canonically be viewed as a three-stage activity (see Figure 8.1) as follows.

Figure 8.1: The three stages of CDPS.



(1) Problem decomposition In this stage, the overall problem to be solved is decomposed into smaller subproblems. The decomposition will typically be hierarchical, so that subproblems are then further decomposed into smaller subproblems, and so on, until the

subproblems are then further decomposed into smaller subproblems, and so on, until the subproblems are of an appropriate granularity to be solved by individual agents. The different levels of decomposition will often represent different levels of problem abstraction. For example, consider a (real-world) example of cooperative problem solving, which occurs when a government body asks whether a new hospital is needed in a particular region. In order to answer this question, a number of smaller subproblems need to be solved, such as whether the existing hospitals can cope, what the likely demand is for hospital beds in the future, and so on. The smallest level of abstraction might involve asking individuals about their day-to-day experiences of the current hospital provision. Each of these different levels in the problem-solving hierarchy represents the problem at a progressively lower level of abstraction.

Notice that the grain size of subproblems is important: one extreme view of CDPS is that a decomposition continues until the subproblems represent ‘atomic’ actions, which cannot be decomposed any further. This is essentially what happens in the ACTOR paradigm, with new agents – ACTORs – being spawned for every subproblem, until ACTORs embody individual program instructions such as addition, subtraction, and so on [Agha, [1986](#)]. But this approach introduces a number of problems. In particular, the overheads involved in managing the interactions between the (typically very many) subproblems outweigh the benefits of a cooperative solution.

Another issue is how to perform the decomposition. One possibility is that the problem is decomposed by one individual agent. However, this assumes that this agent must have the appropriate expertise to do this – it must have knowledge of the *task structure*, that is, how the task is ‘put together’. If other agents have knowledge pertaining to the task structure, then they may be able to assist in identifying a better decomposition. The decomposition itself may therefore be better treated as a cooperative activity.

Yet another issue is that task decomposition cannot in general be done without some knowledge of the agents that will eventually solve problems. There is no point in arriving at a particular decomposition that is impossible for a particular collection of agents to solve.

- (2) **Subproblem solution** In this stage, the subproblems identified during problem decomposition are individually solved. This stage typically involves sharing of information between agents: one agent can help another out if it has information that may be useful to the other.
- (3) **Solution synthesis** In this stage, solutions to individual subproblems are integrated into an overall solution. As in problem decomposition, this stage may be hierarchical, with partial solutions assembled at different levels of abstraction.

Note that the extent to which these stages are explicitly carried out in a particular problem domain will depend very heavily on the domain itself; in some domains, some of the stages might not be present at all.

Given this general framework for CDPS, there are two specific cooperative problem-solving activities that are likely to be present: *task sharing* and *result sharing* [Smith and Davis, [1980](#)] (see [Figure 8.2](#)).

Task sharing takes place when a problem is decomposed to smaller subproblems and allocated to different agents. Perhaps the key problem to be solved in a task-sharing

system is that of how tasks are to be *allocated* to individual agents. If all agents are homogeneous in terms of their capabilities (cf. the discussion on parallel problem solving, above), then task sharing is straightforward: any task can be allocated to any agent. However, in all but the most trivial of cases, agents have very different capabilities. In cases where the agents are really autonomous – and can hence decline to carry out tasks (in systems that do not enjoy the *benevolence* assumption described above), then task allocation will involve agents reaching agreements with others, perhaps by using the negotiation or auction techniques discussed later in this book.

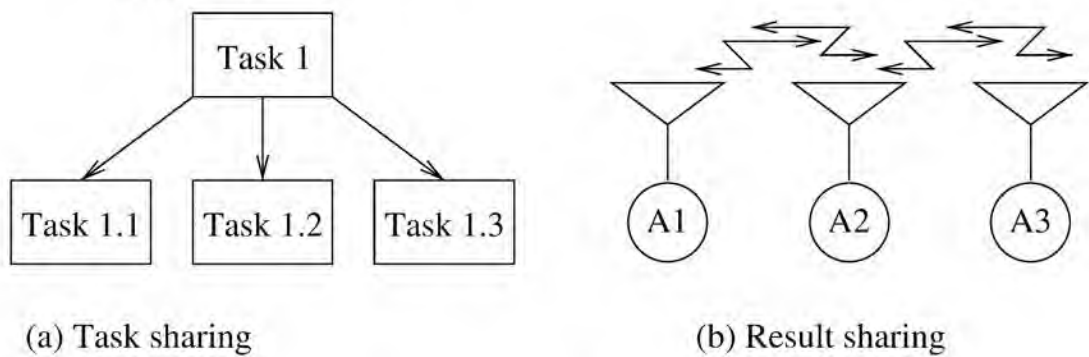
TASK SHARING

Result sharing involves agents sharing information relevant to their subproblems. This information may be shared *proactively* (one agent sends another agent some information because it believes that the other will be interested in it), or *reactively* (an agent sends another information in response to a request that was previously sent – cf. the *subscribe* performatives in the agent communication languages discussed earlier).

RESULT SHARING

In the sections that follow, I shall discuss task sharing and result sharing in more detail.

Figure 8.2: (a) Task sharing and (b) result sharing. In task sharing, a task is decomposed into subproblems that are allocated to agents, while in result sharing, agents supply each other with relevant information, either proactively or on demand.



8.2.1 Task sharing in the Contract Net

The Contract Net (CNET) protocol is a high-level protocol for achieving efficient cooperation through task sharing in networks of communicating problem solvers ([Smith, 1977, 1980a]; [Smith and Davis, 1980]). The basic metaphor used in the CNET is, as the name of the protocol suggests, contracting – Smith took his inspiration from the way that companies organize the process of putting contracts out to tender (see Figure 8.3).

CONTRACT NET

A node that generates a task advertises the existence of that task to other nodes in the net with a *task announcement*, and then acts as the *manager* of that task for its duration. In the absence of any information about the specific capabilities of the other nodes in the net the manager is

of any information about the specific capabilities of the other nodes in the net, the manager is forced to issue a *general broadcast* to all other nodes. If, however, the manager possesses some knowledge about which of the other nodes in the net are likely candidates, then it can issue a *limited broadcast* to just those candidates. Finally, if the manager knows exactly which of the other nodes in the net is appropriate, then it can issue a *point-to-point* announcement. As work on the problem progresses, many such task announcements will be made by various managers.

TASK ANNOUNCEMENT

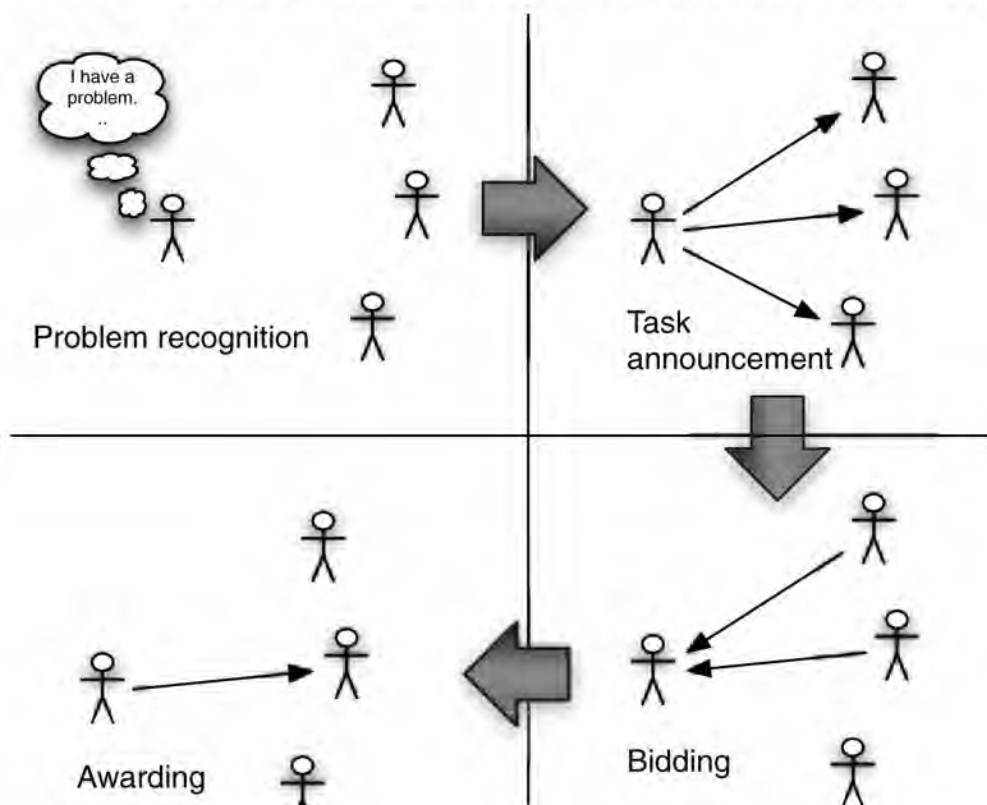
Nodes in the net listen to the task announcements and evaluate them with respect to their own specialized hardware and software resources. When a task to which a node is suited is found, it submits a *bid*. A bid indicates the capabilities of the bidder that are relevant to the execution of the announced task. A manager may receive several such bids in response to a single task announcement; based on the information in the bids, it selects the most appropriate nodes to execute the task. The selection is communicated to the successful bidders through an *award* message. These selected nodes assume responsibility for execution of the task, and each is called a *contractor* for that task.

After the task has been completed, the contractor sends a *report* to the manager. [Smith, 1980b, pp. 60–61]

This normal contract negotiation process can be simplified in some instances, with a resulting enhancement in the efficiency of the protocol. If a manager knows exactly which node is appropriate for the execution of a task, a *directed contract* can be awarded. This differs from the *announced contract* in that no announcement is made and no bids are submitted. Instead, an award is made directly. In such cases, nodes awarded contracts must acknowledge receipt, and have the option of refusal.

DIRECTED CONTRACT

Figure 8.3: The Contract Net (CNET) protocol for task allocation.



Finally, for tasks that amount to simple requests for information, a contract might not be appropriate. In such cases, a request–response sequence can be used without further embellishment. Such messages (that aid in the distribution of data as opposed to control) are implemented as *request* and *information* messages. The request message is used to encode straightforward requests for information when contracting is unnecessary. The information message is used both as a response to a request message and as a general data transfer message [Smith, 1980b, pp. 62–63].

In addition to describing the various messages that agents may send, Smith describes the procedures to be carried out on receipt of a message. Briefly, these procedures are as follows (see [Smith, 1980b, pp. 96–102] for more details).

- *Task announcement processing*

TASK ANNOUNCEMENT PROCESSING

On receipt of a task announcement, an agent decides if it is *eligible* for the task. It does this by looking at the *eligibility specification* contained in the announcement. If it is eligible, then details of the task are stored, and the agent will subsequently bid for the task.

- *Bid processing*

BID PROCESSING

Details of bids from would-be contractors are stored by (would-be) managers until some deadline is reached. The manager then awards the task to a single bidder.

- *Award processing*

AWARD PROCESSING

Agents that bid for a task, but fail to be awarded it, simply delete details of the task. The successful bidder must attempt to expedite the task (which may mean generating new subtasks).

- *Request and inform processing*

AWARD AND INFORM PROCESSING

These messages are the simplest to handle. A request simply causes an inform message to be sent to the requester, containing the required information, but only if that information is immediately available. (Otherwise, the requestee informs the requester that the information is unknown.) An inform message causes its content to be added to the recipient’s database. It is assumed that, at the conclusion of a task, a contractor will send an information message to the manager, detailing the results of the expedited task.¹

How should a potential contractor decide whether or not to bid for a task? The following scheme was proposed in [Sandholm, 1999, pp. 234–235]. Let us suppose that, at time t , a

potential contractor i is currently scheduled to carry out a set of tasks τ_i^t , and that it has an *endowment* of e_i (where this endowment denotes the total financial resource available for agent i in the system). Then i receives an announcement message with task specification ts . Let us denote by $\tau(ts)$ the tasks specified by ts . Further, let $c_i^t(\tau)$ denote the *cost* to agent i of carrying out the set of tasks denoted by τ at time t . Then the *marginal cost* of carrying out tasks τ at time t for agent i is denoted by $\mu_i(\tau(ts) \mid \tau_i^t)$:

MARGINAL COST OF TASK

$$\mu_i(\tau(ts) \mid \tau_i^t) = c_i(\tau(ts) \cup \tau_i^t) - c_i(\tau_i^t).$$

Thus the marginal cost for agent i of agreeing to carry out the task as specified in ts is the difference between the cost of carrying out its currently allocated tasks, and the cost of only carrying out its previously agreed tasks. If $\mu_i(\tau(ts) \mid \tau_i^t) = 0$, for instance, then the marginal (extra) cost of carrying out $\tau(ts)$ is zero: they can be done for ‘free’. If $\mu_i(\tau(ts) \mid \tau_i^t) < e$ (i.e. the total cost of carrying out currently allocated tasks plus those just advertised is less than the agent’s endowment) then it is rational to bid; otherwise, it is not. Note that if the task specification ts contained in the announcement message defines a possible payment for carrying out these tasks (which we denote by $e(ts)$), then the decision as to whether to bid is whether $\mu_i(\tau(ts) \mid \tau_i^t) < (e(ts) + e_i)$: if it is, then bid, otherwise, do not.

Despite (or perhaps because of) its simplicity, the Contract Net has become the most implemented and best-studied framework for distributed problem solving.

8.3 Result Sharing

In result sharing, problem solving proceeds by agents cooperatively exchanging information as a solution is developed. Typically, these results will initially be the solution to small problems, which are then refined into larger, more abstract solutions. [Durfee, 1999, p. 131] suggests that problem solvers can improve group performance in result sharing in the following ways.

Confidence Independently derived solutions can be cross-checked, highlighting possible errors, and increasing confidence in the overall solution.

Completeness Agents can share their *local* views to achieve a better overall *global* view.

Precision Agents can share results to ensure that the precision of the overall solution is increased.

Timeliness Even if one agent could solve a problem on its own, by sharing a solution, the result could be derived more quickly.

8.4 Combining Task and Result Sharing

In the everyday cooperative working that we all engage in, we frequently *combine* task sharing and result sharing. In this section, I will briefly give an overview of how this was achieved in the FELINE system [Wooldridge et al., 1991]. FELINE was a *cooperating expert system*. The

idea was to build an overall problem-solving system as a collection of cooperating experts, each of which had expertise in distinct but related areas. The system worked by these agents cooperating both to *share knowledge* and to *distribute subtasks*. Each agent in FELINE was in fact an independent rule-based system: it had a working memory, or database, containing information about the current state of problem solving; in addition, each agent had a collection of rules, which encoded its domain knowledge.

Each agent in FELINE also maintained a data structure representing its beliefs about itself and its environment. This data structure is called the *environment model* (cf. the agents with symbolic representations discussed in [Chapter 3](#)). It contained an entry for the modelling agent and each agent that the modelling agent might communicate with (its *acquaintances*). Each entry contained two important attributes as follows.

Skills This attribute is a set of identifiers denoting hypotheses which the agent has the expertise to establish or deny. The skills of an agent will correspond roughly to root nodes of the inference networks representing the agent's domain expertise.

Interests This attribute is a set of identifiers denoting hypotheses for which the agent requires the truth value. It may be that an agent actually has the expertise to establish the truth value of its interests, but is nevertheless 'interested' in them. The interests of an agent will correspond roughly to leaf nodes of the inference networks representing the agent's domain expertise.

Messages in FELINE were triples, consisting of *sender*, *receiver*, and *contents*. The contents field was also a triple, containing *message type*, *attribute*, and *value*. Agents in FELINE communicated using three message types as follows (the system predated the KQML and FIPA languages discussed in [Chapter 7](#)).

Request If an agent sends a request, then the attribute field will contain an identifier denoting a hypothesis. It is assumed that the hypothesis is one which lies within the domain of the intended recipient. A request is assumed to mean that the sender wants the receiver to derive a truth value for the hypothesis.

Response If an agent receives a request and manages to successfully derive a truth value for the hypothesis, then it will send a response to the originator of the request. The attribute field will contain the identifier denoting the hypothesis; the value field will contain the associated truth value.

Inform The attribute field of an inform message will contain an identifier denoting a hypothesis. The value field will contain an associated truth value. An inform message will be unsolicited; an agent sends one if it thinks that the recipient will be 'interested' in the hypothesis.

To understand how problem solving in FELINE worked, consider goal-driven problem solving in a conventional rule-based system. Typically, goal-driven reasoning proceeds by attempting to establish the truth value of some hypothesis. If the truth value is not known, then a recursive descent of the inference network associated with the hypothesis is performed. Leaf nodes in the inference network typically correspond to questions which are asked of the user, or data that is acquired in some other way. Within FELINE, this scheme was augmented by the following principle. When evaluating a leaf node, if it was not a question, then the environment model

was checked to see if any other agent had the node as a ‘skill’. If there was some agent that listed the node as a skill, then a request was sent to that agent, requesting the hypothesis. The sender of the request then waited until a response was received; the response indicated the truth value of the node.

Typically, data-driven problem solving proceeds by taking a database of facts (hypotheses and associated truth values), and a set of rules, and repeatedly generating a set of new facts. These new facts are then added to the database, and the process begins again. If a hypothesis follows from a set of facts and a set of rules, then this style of problem solving will eventually generate a result. In FELINE, this scheme was augmented as follows. Whenever a new fact was generated by an agent, the environment model was consulted to see if any agent had the hypothesis as an ‘interest’. If it did, then an ‘inform’ message was sent to the appropriate agent, containing the hypothesis and the truth value. Upon receipt of an ‘inform’ message, the recipient agent added the fact to its database and entered a forward chaining cycle, to determine whether any further information could be derived; this could lead to yet more information being sent to other agents. Similar schemes were implemented in (for example) the CooperA system [Sommaruga et al., [1989](#)].

8.5 Handling Inconsistency

One of the major problems that arise in cooperative activity is that of *inconsistencies* between different agents in the system. Agents may have inconsistencies with respect to both their *beliefs* (the information they hold about the world) and their *goals/intentions* (the things they want to achieve). As I indicated earlier, inconsistencies between goals generally arise because agents are assumed to be autonomous, and thus do not share common objectives.

Inconsistencies between the beliefs that agents have can arise from several sources. First, the viewpoint that agents have will typically be limited – no agent will ever be able to obtain a *complete* picture of their environment. Also, the sensors that agents have may be faulty, or the information sources that the agent has access to may in turn be faulty.

In a system of moderate size, inconsistencies are inevitable: the question is how to deal with them. [Durfee et al., [1989a](#)] suggest a number of possible approaches to the problem as follows.

- Do not allow it to occur – or at least ignore it. This is essentially the approach of the Contract Net: task sharing is always driven by a manager agent, who has the only view of the problem that matters.
- Resolve inconsistencies through bargaining. While this may be desirable in theory, the communication and computational overheads incurred suggest that it will rarely be possible in practice.
- Build systems that degrade gracefully in the presence of inconsistency.

The third approach is clearly the most desirable. [Lesser and Corkill, [1981](#)] refer to systems that can behave robustly in the presence of inconsistency as *functionally accurate/cooperative* (FA/C):

[FA/C](#)

[In FA/C systems] ... nodes cooperatively exchange and integrate partial, tentative, high-level results to construct a consistent and complete solution. [An agent’s] problem solving is structured so that its local knowledge bases need not be complete, consistent.

and up-to-date in order to make progress on its problem-solving tasks. Nodes do the best they can with their current information, but their solutions to their local subproblems may be only partial, tentative, and incorrect.

[Durfee et al., [1989a](#), pp. 117–118]

[Lesser and Corkill, [1981](#)] suggested the following characteristics of FA/C systems that tolerate inconsistent/incorrect information.

- Problem solving is not tightly constrained to a particular sequence of events – it progresses opportunistically (i.e. not in a strict predetermined order, but taking advantage of whatever opportunities arise) and incrementally (i.e. by gradually piecing together solutions to subproblems).
- Agents communicate by exchanging high-level intermediate results, rather than by exchanging raw data.
- Uncertainty and inconsistency is implicitly resolved when partial results are exchanged and compared with other partial solutions. Thus inconsistency and uncertainty are resolved as problem solving progresses, rather than at the beginning or end of problem solving.
- The solution is not constrained to a single solution route: there are many possible ways of arriving at a solution, so that if one fails, there are other ways of achieving the same end. This makes the system robust against localized failures and bottlenecks in problem solving.

8.6 Coordination

Perhaps the defining problem in cooperative working is that of *coordination*. The coordination problem is that of *managing inter-dependencies between the activities of agents*: some coordination mechanism is essential if the activities that agents can engage in can interact in any way. How might two activities interact? Consider the following real-world examples.

COORDINATION

- You and I both want to leave the room, and so we independently walk towards the door, which can only fit one of us. I graciously permit you to leave first.

In this example, our activities need to be coordinated because there is a resource (the door) which we both wish to use, but which can only be used by one person at a time.

- I intend to submit a grant proposal, but in order to do this, I need your signature.

In this case, my activity of sending a grant proposal depends upon your activity of signing it off – I cannot carry out my activity until yours is completed. In other words, my activity *depends* upon yours.

- I obtain a soft copy of a paper from a web page. I know that this report will be of interest to you as well. Knowing this, I proactively photocopy the report, and give you a copy.

In this case, our activities do not strictly need to be coordinated – since the report is freely available on a web page, you could download and print your own copy. But by proactively printing a copy, I save you time and hence, intuitively, increase your utility.

[von Martini [1990](#)] suggested a typology for coordination relationships (see [Figure 8.4](#)). He

[von Martial, 1990] suggested a typology for coordination relationships (see [Figure 8.4](#)). He suggested that, broadly, relationships between activities could be either *positive* or *negative*.

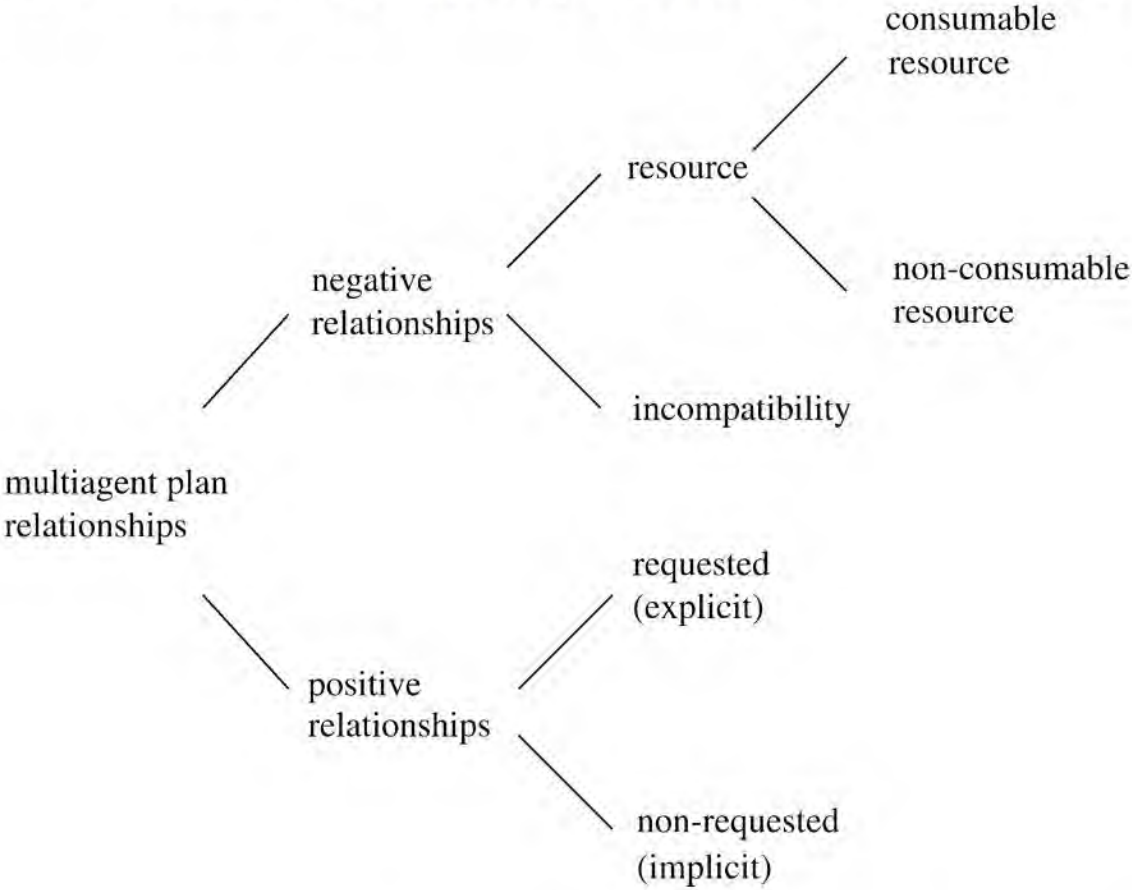
COORDINATION RELATIONSHIPS

Positive relationships ‘are all those relationships between two plans from which some benefit can be derived, for one or both of the agents’ plans, by combining them’ [von Martial, 1990, p. 111]. Such relationships may be *requested* (I *explicitly* ask you for help with my activities) or *non-requested* (it so happens that, by working together, we can achieve a solution that is better for at least one of us, without making the other any worse off). [von Martial, 1990, p. 112] distinguishes three types of non-requested relationship as follows.

The action equality relationship We both plan to perform an identical action, and by recognizing this, one of us can perform the action alone and so save the other effort.

ACTION EQUALITY

Figure 8.4: Von Martial’s typology of coordination relationships.



The consequence relationship The actions in my plan have the side-effect of achieving one of your goals, thus relieving you of the need to explicitly achieve it.

CONSEQUENCE

The favour relationship Some part of my plan has the side effect of contributing to the achievement of one of your goals, perhaps by making it easier (e.g. by achieving a precondition of one of the actions in it).

FAVOUR

Coordination in multiagent systems is assumed to happen *at run-time*; that is, the agents themselves must be capable of recognizing these relationships and, where necessary, managing them as part of their activities [von Martial, 1992]. This contrasts with the more conventional situation in computer science, where a designer explicitly attempts to anticipate possible interactions in advance, and designs the system so as to avoid negative interactions and exploit potential positive interactions.

In the sections that follow, I present some of the main approaches that have been developed for dynamically coordinating activities.

8.6.1 Coordination through partial global planning

The Distributed Vehicle Monitoring Testbed (DVMT) was one of the earliest and best-known testbeds for multiagent systems. The DVMT was a fully instrumented testbed for developing distributed problem-solving networks [Lesser and Corkill, 1988; Lesser and Erman, 1980]. The testbed was based around the domain of distributed vehicle sensing and monitoring, the aim being to successfully track a number of vehicles that pass within the range of a set of distributed sensors. The main purpose of the testbed was to support experimentation into different problem-solving strategies.

DVMT

The distributed sensing domain is inherently data driven: new data about vehicle movements appears and must be processed by the system. The main problem with the domain was to process information as rapidly as possible, so that the system could come to conclusions about the paths of vehicles in time for them to be useful. To coordinate the activities of agents in the DVMT, Durfee developed an approach known as *partial global planning* [Durfee, 1988, 1996; Durfee and Lesser, 1987].

PARTIAL GLOBAL PLANNING

The main principle of partial global planning is that cooperating agents exchange information in order to reach common conclusions about the problem-solving process. Planning is *partial* because the system does not (indeed *cannot*) generate a plan for the entire problem. It is *global* because agents form non-local plans by exchanging local plans and cooperating to achieve a non-local view of problem solving.

Partial global planning involves three iterated stages.

1. Each agent decides what its own goals are and generates short-term plans in order to achieve them.
2. Agents exchange information to determine where plans and goals interact.
3. Agents alter local plans in order to better coordinate their own activities.

In order to prevent incoherence during these processes, Durfee proposed the use of a *meta-level structure*, which guided the cooperation process within the system. The meta-level structure dictates which agents an agent should exchange information with, and under what conditions they should do so.

The actions and interactions of a group of agents are incorporated into a data structure known as a *partial global plan* (PGP). This data structure is generated cooperatively by agents

exchanging information. It contains the following principal attributes:

Objective The objective is the larger goal that the system is working towards.

Activity maps An activity map is a representation of what agents are actually doing, and what results will be generated by their activities.

Solution construction graph A solution construction graph is a representation of how agents should interact, and what information should be exchanged, and when, in order for the system to successfully generate a result.

Keith Decker extended and refined the PGP coordination mechanisms in his TÆMS testbed [Decker, [1996](#)]; this led to what he called *generalized partial global planning* (GPGP) [Decker and Lesser, [1995](#)]. GPGP makes use of five techniques for coordinating activities:

GPGP

Updating non-local viewpoints Agents have only local views of activity, and so sharing information can help them achieve broader views. In his TÆMS system, Decker uses three variations of this policy: communicate no local information, communicate all information, or an intermediate level.

Communicating results Agents may communicate results in three different ways. A minimal approach is where agents only communicate results that are essential to satisfy obligations. Another approach involves sending all results. A third is to send results to those with an interest in them.

Handling simple redundancy Redundancy occurs when efforts are duplicated. This may be deliberate – an agent may get more than one agent to work on a task because it wants to ensure that the task gets done. However, in general, redundancies indicate wasted resources, and are therefore to be avoided. The solution adopted in GPGP is as follows. When redundancy is detected, in the form of multiple agents working on identical tasks, one agent is selected at random to carry out the task. The results are then broadcast to other interested agents.

Handling hard coordination relationships ‘Hard’ coordination relationships are essentially the ‘negative’ relationships of von Martial, as discussed above. Hard coordination relationships are thus those that threaten to prevent activities being successfully completed. Thus a hard relationship occurs when there is a danger of the agents’ actions destructively interfering with one another, or preventing each other’s actions being carried out. When such relationships are encountered, the activities of agents are rescheduled to resolve the problem.

Handling soft coordination relationships ‘Soft’ coordination relationships include the ‘positive’ relationships of von Martial. Thus these relationships include those that are not ‘mission critical’, but that may improve overall performance. When these are encountered, then rescheduling takes place, but with a high degree of ‘negotiability’: if rescheduling is not found possible, then the system does not worry about it too much.

8.6.2 Coordination through joint intentions

The second approach to coordination that I shall discuss is the use of *human teamwork models*. We saw in [Chapter 4](#) how some researchers have built agents around the concept of practical reasoning, and how central intentions are in this practical reasoning process. Intentions also play a critical role in coordination: they provide both the stability and the predictability that are necessary for social interaction, and the flexibility and reactivity that are necessary to cope with a changing environment. If you know that I am planning to write a book, for example, then this gives you information that you can use to coordinate your activities with mine. For example, it allows you to rule out the possibility of going on holiday with me, or partying with me all night, because you know I will be working hard on the book. When humans work together as a team, mental states that are closely related to intentions appear to play a similarly important role [Cohen and Levesque, [1991](#); Levesque et al., [1990](#)]. It is important to be able to distinguish coordinated action that is not cooperative from coordinated cooperative action. As an illustration of this point, consider the following scenario [Searle, [1990](#)].

A group of people are sitting in a park. As a result of a sudden downpour all of them run to a tree in the middle of the park because it is the only available source of shelter. This may be coordinated behaviour, but it is not cooperative action, as each person has the intention of avoiding becoming wet, and even if they are aware of what others are doing and what their goals are, it does not affect their intended action. This contrasts with the situation in which the people are dancers, and the choreography calls for them to converge on a common point (the tree). In this case, the individuals are performing exactly the same actions as before, but because they each have the aim of meeting at the central point as a consequence of the overall aim of executing the dance, this is cooperative action.

How does having an individual intention towards a particular goal differ from being part of a team, with some sort of collective intention towards the goal? The distinction was first studied in [Levesque et al., [1990](#)], where it was observed that being part of a team implies some sort of *responsibility* towards the other members of the team. To illustrate this, suppose that you and I are together lifting a heavy object as part of a team activity. Then clearly we both individually have the intention to lift the object – but is there more to teamwork than this? Well, suppose I come to believe that it is not going to be possible to lift it for some reason. If I just have an *individual* goal to lift the object, then the rational thing for me to do is simply drop the intention (and thus perhaps also the object). But you would hardly be inclined to say that I was cooperating with you if I did so. Being part of a team implies that I show some responsibility towards you: that if I discover the team effort is not going to work, then I should at least attempt to make you aware of this.

Building on the work of [Levesque et al., [1990](#)], Jennings distinguished between the *commitment* that underpins an intention and the associated *convention* [Jennings, [1993a](#)]. A *commitment* is a pledge or a promise (for example, to have lifted the object); a *convention* in contrast is a means of monitoring a commitment – it specifies under what circumstances a commitment can be abandoned and how an agent should behave both locally and towards others when one of these conditions arises.

COMMITMENT CONVENTION

In more detail, one may commit either to a particular course of action, or, more generally, to a state of affairs. Here, we are concerned only with commitments that are *future directed* towards a state of affairs. Commitments have a number of important properties (see [Jennings, 1993a] and [Cohen and Levesque, 1990a, pp. 217–219] for a discussion), but the most important is that *commitments persist*: having adopted a commitment, we do not expect an agent to drop it until, for some reason, it becomes redundant. The conditions under which a commitment can become redundant are specified in the associated convention – examples include the motivation for the goal no longer being present, the goal being achieved, and the realization that the goal will never be achieved [Cohen and Levesque, 1990a].

When a group of agents are engaged in a cooperative activity they must have a joint commitment to the overall aim, as well as their individual commitments to the specific tasks that they have been assigned. This joint commitment shares the persistence property of the individual commitment; however, it differs in that its state is distributed among the team members. An appropriate social convention must also be in place. This social convention identifies the conditions under which the joint commitment can be dropped, and also describes how an agent should behave towards its fellow team members. For example, if an agent drops its joint commitment because it believes that the goal will never be attained, then it is part of the notion of ‘cooperativeness’ that is inherent in joint action that it informs all of its fellow team members of its change of state. In this context, social conventions provide general guidelines and a common frame of reference in which agents can work. By adopting a convention, every agent knows what is expected both of it and of every other agent as part of the collective working towards the goal, and knows that every other agent has a similar set of expectations.

We can begin to define this kind of cooperation in the notion of a *joint persistent goal* (JPG), as defined in [Levesque et al., 1990]. In a JPG, a group of agents have a collective commitment to bringing about some goal ϕ ; the *motivation* for this goal, i.e. the reason that the group has the commitment, is represented by ψ . Thus ϕ might be ‘move the heavy object’, while ψ might be ‘Michael wants the heavy object moved’. The mental state of the team of agents with this JPG might be described as follows:

MOTIVATION

- initially, every agent does not believe that the goal ϕ is satisfied, but believes ϕ is possible
- every agent i then has a goal of ϕ until the termination condition is satisfied (see below)
- until the termination condition is satisfied, then
 - if any agent i believes that the goal is achieved, then it will have a goal that this becomes a mutual belief, and will retain this goal until the termination condition is satisfied
 - if any agent i believes that the goal is impossible, then it will have a goal that this becomes a mutual belief, and will retain this goal until the termination condition is satisfied
 - if any agent i believes that the motivation ψ for the goal is no longer present, then it will have a goal that this becomes a mutual belief and will retain this goal until

the termination condition is satisfied

- the termination condition is that it is mutually believed that either
 - the goal φ is satisfied
 - the goal φ is impossible to achieve
 - the motivation/justification ψ for the goal is no longer present.

Commitments and conventions in ARCHON

[Jennings, 1993a, 1995] investigated the use of commitments such as JPGs in the coordination of an industrial control system called ARCHON [Jennings et al., 1996a; Perriolat et al., 1996; Wittig, 1992]. He noted that commitments and conventions could be encoded as *rules* in a rule-based system. This makes it possible to explicitly encode coordination structures in the reasoning mechanism of an agent.

ARCHON agents have three layers. The lowest layer is the *control* layer. This layer contains domain-specific agent capabilities. The idea is that agents in ARCHON *wrap* legacy software in agent-like capabilities. In the ARCHON case, these legacy systems were stand-alone expert systems. The legacy systems were embedded within a control module, which provided access to them via an API. ARCHON agents maintained three different types of information, in the forms of an *acquaintance model* (cf. the acquaintance models of the MACE system described later in this chapter), a *self model* (which contains information about the agent's own skills and interests), and finally a general-purpose information store, which contains other information about the agent's environment. The behaviour of the agent was determined by three main control modules: the *cooperation module*, which was responsible for the agent's social ability; the *situation assessment module*, which was responsible for determining when the need for new teamwork arose; and the *communication manager*, which was responsible for sending/receiving messages.

Some of the rules used for reassessing joint commitments and selecting actions to repair failed teamwork are shown in [Figure 8.5](#) (from [Jennings, 1995]). The first four rules capture the conditions where the joint goal has been successfully achieved, where the motivation for the joint goal is no longer present, and where the current plan to achieve the joint goal has been invalidated in some way. The following 'select' rules are used to determine a repair action.

Milind Tambe developed a similar framework for teamwork called Steam [Tambe, 1997]. Agents in Steam are programmed using the Soar rule-based architecture [Newell, 1990; Newell et al., 1989]. The cooperation component of Steam is encoded in about 300 domain-independent rules, somewhat similar in principle to Jennings's teamwork rules, as shown in [Figure 8.5](#). However, the cooperation rules of Steam are far more complex, allowing for sophisticated hierarchical team structures.

The Steam framework was used in a number of application domains, including military mission simulations, as well as the RoboCup simulated robotic soccer domain.

A teamwork-based model of CDPS

Building on Jennings's teamwork-based coordination model [Jennings, 1995], a four-stage

model of CDPS was presented in [Wooldridge and Jennings, 1994, 1999]. The four stages of the model are as follows.

- (1) **Recognition** CDPS begins when some agent in a multiagent community has a goal, and recognizes the potential for cooperative action with respect to that goal. Recognition may occur for several reasons. The paradigm case is that in which the agent is unable to achieve the goal in isolation, but believes that cooperative action can achieve it. For example, an agent may have a goal which, to achieve it, requires information that is only accessible to another agent. Without the cooperation of this other agent, the goal cannot be achieved. More prosaically, an agent with a goal to move a heavy object might simply not have the strength to do this alone.

Figure 8.5: Joint commitment rules in ARCHON.

Match rules:

- R1: if *task t has finished executing and
 t has produced desired outcome of joint action*
 then *joint goal is satisfied.*
- R2: if *receive information i and
 i is related to triggering conditions
 for joint goal G and
 i invalidates beliefs for wanting G*
 then *motivation for G is no longer present.*
- R3: if *delay task t1 and
 t1 is a component of common recipe R and
 t1 must be synchronized with t2 in R*
 then *R is violated.*
- R4: if *finished executing common recipe R and
 expected results of R not produced and
 alternative recipe exists*
 then *R is invalid.*

Select rules:

- R1: if *joint goal is satisfied*
 then *abandon all associated local activities and
 inform cooperation module.*
- R2: if *motivation for joint goal no longer present*
 then *abandon all associated local activities and
 inform cooperation module.*
- R3: if *common recipe R is violated and
 R can be rescheduled*
 then *suspend local activities associated with R and
 reset timings and descriptions associated with R and
 inform cooperation module.*
- R4: if *common recipe R1 is invalid and
 alternative recipe R2 exists*
 then *abandon all local activities with R1 and
 inform cooperation module that R1 is invalid and
 propose R2 to cooperation module.*

Alternatively, an agent may be able to achieve the goal on its own, but might not want to. There may be several reasons for this. First, it may believe that, in working alone, it will clobber one of its other goals. For example, suppose I have a goal of lifting a heavy object. I may have the capability of lifting the object, but I might believe that, in so doing, I would injure my back, thereby clobbering my goal of being healthy. In this case, a cooperative solution – involving no injury to my back – is preferable. More generally, an agent may believe that a cooperative solution will in some way be better than a solution achieved by action in isolation. For example, a solution might be obtained more quickly, or may be more accurate, as a result of cooperative action.

Believing that you either cannot achieve your goal in isolation, or (for whatever reason) preferring not to work alone, is part of the potential for cooperation. But it is not enough in itself to initiate the social process. For there to be potential for cooperation with respect to an agent's goal, the agent must also believe that there is some group of agents that can actually achieve the goal.

- (2) **Team formation** During this stage, the agent that recognized the potential for cooperative action at stage (1) solicits assistance. If this stage is successful, then it will end with a group of agents having some kind of nominal commitment to collective action. This stage is essentially a collective deliberation stage. At the conclusion of this stage, the team will have agreed on the *ends* to be achieved (i.e. the principle of joint action), but not on the means (i.e. the way in which this end will be achieved). Note that the agents are assumed to be rational, in the sense that they will not form a team unless they implicitly believe that the goal is achievable.
- (3) **Plan formation** We saw above that a group will not form a collective unless they believe that they can actually achieve the desired goal. This, in turn, implies that there is at least one action known to the group that will take them 'closer' to the goal. However, it is possible that there are many agents that know of actions that the group can perform in order to take them closer to the goal. Moreover, some members of the collective may have objections to one or more of these actions. It is therefore necessary for the collective to come to some agreement about exactly which course of action they will follow. Such an agreement may be reached via negotiation, as discussed later in this book.
- (4) **Team action** During this stage, the newly agreed plan of joint action is executed by the agents, which maintain a close-knit relationship throughout. This relationship is defined by a convention, which every agent follows. The JPG described above might be one possible convention.

8.6.3 Coordination by mutual modelling

Another approach to coordination, closely related to the models of human teamwork discussed above, is that of *coordination by mutual modelling*. The idea is as follows. Recall the simple coordination example I gave earlier: you and I are both walking to the door, and there is not enough room for both of us – a collision is imminent. What should we do? One option is for both of us to simply stop walking. This possibility guarantees that no collision will occur, but it is in some sense suboptimal: while we stand and wait, there is an unused resource (the door), which could fruitfully have been exploited by one of us. Another

possibility is for both of us to *put ourselves in the place of the other*: to build a model of other agents – their beliefs, intentions, and the like – and to coordinate our activities around the predictions that this model makes. In this case, you might believe that I am eager to please you, and therefore that I will probably allow you to pass through the door first; on this basis, you can continue to walk to the door.

MACE

Les Gasser's MACE system, developed in the mid 1980s, can, with some justification, claim to be the first general experimental testbed for multiagent systems [Gasser et al., [1987b](#)]. MACE is noteworthy for several reasons, but perhaps most importantly because it brought together most of the components that have subsequently become common in testbeds for developing multiagent systems. I mention it in this section because of one critical component: the *acquaintance models*, which are discussed in more detail below. Acquaintance models are representations of other agents: their abilities, interests, capabilities, and the like.

ACQUAINTANCE MODELS

A MACE system contains five components:

- a collection of *application agents*, which are the basic computational units in a MACE system (see below)
- a collection of predefined *system agents*, which provide service to users (e.g. user interfaces)
- a collection of facilities, available to all agents (e.g. a pattern matcher)
- a *description database*, which maintains agent descriptions, and produces executable agents from those descriptions
- a set of *kernels*, one per physical machine, which handle communication and message routing, etc.

Gasser et al. identified three aspects of agents: they contain knowledge, they sense their environment, and they perform actions [Gasser et al., [1987b](#), p. 124]. Agents have two kinds of knowledge: specialized, local, domain knowledge, and *acquaintance knowledge* – knowledge about other agents. An agent maintains the following information about its acquaintances [Gasser et al., [1987b](#), pp. 126–127].

Class Agents are organized in structured groups called *classes*, which are identified by a class name.

Name Each agent is assigned a name, unique to its class – an agent's address is a (*class, name*) pair.

Roles A role describes the part an agent plays in a class.

Skills Skills are what an agent knows about the capabilities of the modelled agent.

Goals Goals are what the agent knows the modelled agent wants to achieve.

Plans Plans are an agent's view of the way a modelled agent will achieve its goals.

Figure 8.6: Structure of MACE agents.

```
( (NAME plus-ks)
  (IMPORT ENGINE FROM dbb-def)
  (ACQUAINTANCES
    (plus-ks
      ... model for plus-ks ...
    )
    (de-exp
      [ROLE (ORG-MEMBER)]
      [GOALS ( ... goal list ... )]
      [SKILLS ( ... skill list ... )]
      [PLANS ( ... plan list ...)]
    )
    (simple-plus
      ... acquaintance model for simple-plus ...
    )
  )
  (INIT-CODE ( ... LISP code ...))
) ; end of plus-ks
```

Agents sense their environment primarily through receiving messages. An agent's ability to act is encoded in its *engine*. An engine is a LISP function, evaluated by default once on every scheduling cycle. The only externally visible signs of an agent's activity are the messages it sends to other agents. Messages may be directed to a single agent, a group of agents, or all agents; the interpretation of messages is left to the programmer to define.

An example MACE agent is shown in [Figure 8.6](#). The agent modelled in this example is part of a simple calculator system implemented using the blackboard model. The agent being modelled here is called PLUS-KS. It is a knowledge source that knows about how to perform the addition operation. The PLUS-KS knowledge source is the 'parent' of two other agents; DE-EXP, an agent that knows how to decompose simple expressions into their primitive components, and SIMPLE-PLUS, an agent that knows how to add two numbers.

The definition frame for the PLUS-KS agent consists of a name for the agent – in this case PLUS-KS – the engine, which defines what actions the agent may perform (in this case the engine is imported, or inherited, from an agent called DBB-DEF), and the acquaintances of the agent.

The acquaintances slot for PLUS-KS defines models for three agents. Firstly, the agent models itself. This defines how the rest of the world will see PLUS-KS. Next, the agents DE-EXP and SIMPLE-PLUS are modelled. Consider the model for the agent DE-EXP. The role slot defines the relationship of the modelled agent to the modeller. In this case, both DE-EXP and SIMPLE-PLUS are members of the class defined by PLUS-KS. The GOALS slot defines what the modelling agent believes the modelled agent wants to achieve. The SKILLS slot defines what resources the modeller believes the modelled agent can provide. The PLANS slot defines how the modeller believes the modelled agent will achieve its goals. The PLANS slot consists of a list of skills, or operations, which the modelled agent will perform in order to achieve its goals.

Gasser et al. described how MACE was used to construct blackboard systems, a Contract Net system, and a number of other experimental systems (see [Gasser et al., [1987b](#), [1989](#)]).

8.6.4 Coordination by norms and social laws

In our everyday lives, we use a range of techniques for coordinating activities. One of the most important is the use of *norms* and *social laws* [Lewis, [1969](#)]. A norm is simply an established, expected pattern of behaviour; the term social law carries essentially the same meaning, but it is usually implied that social laws carry with them some authority. Examples of norms in human society abound. For example, in the UK, it is a norm to form a queue when waiting for a bus, and to allow those who arrived first to enter the bus first. This norm is not *enforced* in any way; it is simply expected behaviour, and diverging from this norm will (usually) cause nothing more than icy looks from others on the bus. Nevertheless, this norm provides a template that can be used by all those around to regulate their own behaviour.

SOCIAL LAWS

Conventions play a key role in the social process. They provide agents with a template upon which to structure their action repertoire. They represent a behavioural constraint, striking a balance between individual freedom on the one hand, and the goal of the agent society on the other. As such, they also simplify an agent's decision-making process, by dictating courses of action to be followed in certain situations. It is important to emphasize what a key role conventions play in our everyday lives. As well as formalized conventions, which we all recognize as such (an example being driving on the left- or right-hand side of the road), almost every aspect of our social nature is dependent on convention. After all, language itself is nothing more than a convention, which we use in order to coordinate our activities with others.

One key issue in the understanding of conventions is to decide on the most effective method by which they can come to exist within an agent society. There are two main approaches as follows.

Offline design In this approach, social laws are designed offline, and hardwired into agents. Examples in the multiagent systems literature include [Conte and Castelfranchi, [1993](#); Goldman and Rosenschein, [1994](#); Shoham and Tennenholtz, [1992b](#)].

Emergence from within the system This possibility was investigated by [Kittock, [1993](#); Shoham and Tennenholtz, [1992a](#); Walker and Wooldridge, [1995](#)], who experimented with a number of techniques by which a convention can 'emerge' from within a group of agents.

The first approach will often be simpler to implement, and might present the system designer with a greater degree of control over system functionality. However, there are a number of disadvantages with this approach. First, it is not always the case that *all* the characteristics of a system are known at design time. (This is most obviously true of open systems such as the Internet.) In such systems, the ability of agents to organize themselves would be advantageous. Secondly, in complex systems, the goals of agents (or groups of agents) might be constantly changing. To keep reprogramming agents in such circumstances would be costly and inefficient. Finally, the more complex a system becomes, the less likely it is that system designers will be able to design effective norms or social laws: the dynamics of the system – the possible 'trajectories' that it can take – will be too hard to predict. Here, flexibility within the agent society might result in greater coherence.

Emergent norms and social laws

A key issue, then, is how a norm or social law can emerge in a society of agents. In particular, the question of how agents can come to reach a *global* agreement on the use of social conventions by using only *locally available* information is of critical importance. The convention must be *global* in the sense that all agents use it. But each agent must decide on which convention to adopt based solely on its own experiences, as recorded in its internal state; predefined interagent power structures or authority relationships are not allowed.

This problem was perhaps first investigated in [Shoham and Tennenholtz, [1992a](#)], who considered the following scenario, which I will call the *tee shirt game*.

Consider a group of agents, each of which has two tee shirts: one red and one blue. The agents – who have never met previously, and who have no prior knowledge of each other – play a game, the goal of which is for *all* the agents to end up wearing the same-coloured tee shirt. Initially, each agent wears a red or blue tee shirt selected randomly. The game is played in a series of rounds. On each round, every agent is paired up with exactly one other agent; pairs are selected at random. Each pair gets to see the colour of the tee shirt the other is wearing – no other information or communication between the agents is allowed. After a round is complete, every agent is allowed either to stay wearing the same coloured tee shirt, or to swap to the other colour.

Notice that no global view is possible in this game: an agent can never ‘climb the wall’ to see what every other agent is wearing. An agent must therefore base its decision about whether to change tee shirts or stick with the one it is currently wearing using only its memory of the agents it has encountered on previous rounds. The key problem is this: to design what [Shoham and Tennenholtz, [1992b](#)] refer to as a *strategy update function*, which represents an agent’s decision-making process. A strategy update function is a function from the history that the agent has observed so far, to a colour (red or blue). Note that the term ‘strategy’ here may be a bit misleading – it simply refers to the colour of the tee shirt. The goal is to develop a strategy update function that, when it is used by every agent in the society, will bring the society to a global agreement as efficiently as possible.

STRATEGY UPDATE FUNCTION

In [Shoham and Tennenholtz, [1992b](#), [1997](#); Walker and Wooldridge, [1995](#)], a number of different strategy update functions were evaluated as follows.

Simple majority This is the simplest form of update function. Agents will change to an alternative strategy if, so far, they have observed more instances of it in other agents than their present strategy. If more than one strategy has been observed more than that currently adopted, the agent will choose the strategy observed most often.

Simple majority with agent types This is as simple majority, except that agents are divided into two types. As well as observing each other’s strategies, agents in these experiments can communicate with others whom they can ‘see’, and who are of the same type. When they communicate, they exchange memories, and each agent treats the other agent’s memory as if it were his own, thus being able to take advantage of another agent’s experiences. In other words, agents are particular about whom they

confide in.

Simple majority with communication on success This strategy updates a form of communication based on a success threshold. When an individual agent has reached a certain level of success with a particular strategy, it communicates its memory of experiences with this successful strategy to all other agents that it can ‘see’. Note that only the memory relating to the successful strategy is broadcast, not the whole memory. The intuition behind this update function is that an agent will only communicate with another agent when it has something *meaningful* to say. This prevents ‘noise’ communication.

Highest cumulative reward For this update to work, an agent must be able to see that using a particular strategy gives a particular pay-off. The highest cumulative reward update rule then says that an agent uses the strategy that it sees has resulted in the highest cumulative pay-off to date.

In addition, the impact of *memory restarts* on these strategies was investigated. Intuitively, a memory restart means that an agent periodically ‘forgets’ everything it has seen to date – its memory is emptied, and it starts as if from scratch again. The intuition behind memory restarts is that it allows an agent to avoid being over-committed to a particular strategy as a result of history: memory restarts thus make an agent more ‘open to new ideas’.

The *efficiency of convergence* was measured by [Shoham and Tennenholtz, 1992b] primarily by the *time taken to convergence*: how many rounds of the tee shirt game need to be played before all agents converge on a particular strategy. However, it was noted in [Walker and Wooldridge, 1995] that *changing* from one strategy to another can be expensive. Consider a strategy such as using a particular kind of computer operating system. Changing from one to another has an associated cost, in terms of the time spent to learn it, and so we do not wish to change too frequently. Another issue is that of *stability*. We do not usually want our society to reach agreement on a particular strategy, only for it then to immediately fall apart, with agents reverting to different strategies.

When evaluated in a series of experiments, all of the strategy update functions described above led to the emergence of particular conventions within an agent society. However, the most important results were associated with the highest cumulative reward update function [Shoham and Tennenholtz, 1997, pp. 150–151]. It was shown that, for any value ϵ such that $0 < \epsilon \leq 1$, there exists some bounded value n such that a collection of agents using the highest cumulative reward update function will reach agreement on a strategy in n rounds with probability $1 - \epsilon$. Furthermore, it was shown that this strategy update function is stable in the sense that, once reached, the agents would not diverge from the norm. Finally, it was shown that the strategy on which agents reached agreement was ‘efficient’, in the sense that it guarantees agents a pay-off no worse than that they would have received had they stuck with the strategy they initially chose.

Offline design of norms and social laws

The alternative to allowing conventions to emerge within a society is to design them offline, before the multiagent system begins to execute. There have been several studies of offline design of social laws, particularly with respect to the computational complexity of the social law design problem [Shoham and Tennenholtz, 1992b, 1996]. To understand the way these

problems are formulated, recall the way in which agents were defined in [Chapter 2](#), as functions from runs (which end in environment states) to actions:

$$Ag: R^E \rightarrow Ac.$$

A *constraint* is then a pair

$$(E', \alpha),$$

where

- $E' \subseteq E$ is a set of environment states
- $\alpha \in Ac$ is an action.

The reading of a constraint (E', α) is that, if the environment is in some state $e \in E'$, then the action α is forbidden. A *social law* is then defined to be a set sl of such constraints. An agent – or plan, in the terminology of [Shoham and Tennenholtz, [1992b](#), p. 279] – is then said to be legal with respect to a social law sl if it never attempts to perform an action that is forbidden by some constraint in sl .

The next question is to define what is meant by a *useful* social law. The answer is to define a set $F \subseteq E$ of *focal states*. The intuition here is that these are the states that are *always legal*, in that an agent should always be able to ‘visit’ the focal states. To put it another way, whenever the environment is in some focal state $e \in F$, it should be possible for the agent to act so as to be able to guarantee that any other state $e' \in F$ is brought about. A useful social law is then one that does not constrain the actions of agents so as to make this impossible.

USEFUL SOCIAL LAW

FOCAL STATE

The *useful social law problem* can then be understood as follows.

USEFUL SOCIAL LAW PROBLEM

Given an environment $Env = (E, \tau, e_0)$ and a set of focal states $F \subseteq E$, find a useful social law, if one exists, or else announce that none exists.

In [Shoham and Tennenholtz, [1992b](#), [1996](#)], it is proved that this problem is NP-complete, and so is unlikely to be soluble by ‘normal’ computing techniques in reasonable time. Some variations of the problem are discussed in [Shoham and Tennenholtz, [1992b](#), [1996](#)], and some cases where the problem becomes tractable are examined. However, these tractable instances do not appear to correspond to useful real-world cases.

Social laws in practice

Before leaving the subject of social laws, I will briefly discuss some examples of social laws that have been evaluated both in theory and in practice. These are *traffic laws* [Shoham and Tennenholtz, [1996](#)].

Imagine a two-dimensional grid world – rather like the Tileworld introduced in [Chapter 2](#) – populated by mobile robots. Only one robot is allowed to occupy a grid point at any one time – more than one is a collision. The robots must collect and transport items from one

grid point to another. The goal is then to design a social law that prevents collisions. However, to be useful in this setting, the social law must not impede the movement of the robots to such an extent that they are unable to get from a location where they collect an item to the delivery location. As a first cut, consider a law which completely constrains the movements of robots, so that they must all follow a single, completely predetermined path, leaving no possibility of collision. Here is an example of such a social law, from [Shoham and Tennenholtz, 1996, p. 602].

Each robot is required to move constantly. The direction of motion is fixed as follows. On even rows each robot must move left, while in odd rows it must move right. It is required to move up when it is in the rightmost column. Finally, it is required to move down when it is on either the leftmost column of even rows or the second rightmost column of odd rows. The movement is therefore in a ‘snake-like’ structure, and defines a Hamiltonian cycle on the grid.

It should be clear that, using this social law,

- the next move of an agent is *uniquely* determined: the law does not leave any doubt about the next state to move to
- an agent will always be able to get from its current location to its desired location
- to get from the current location to the desired location will require at most $O(n^2)$ moves, where n is the size of the grid (to see this, simply consider the dimensions of the grid).

Although it is effective, this social law is obviously not very efficient: surely there are more ‘direct’ social laws which do not involve an agent moving around all the points of the grid? [Shoham and Tennenholtz, 1996] give an example of one, which superimposes a ‘road network’ on the grid structure, allowing robots to change direction as they enter a road. They show that this social law guarantees to avoid collisions, while permitting agents to achieve their goals much more efficiently than the naive social law described above.

8.7 Multiagent Planning and Synchronization

An obvious issue in multiagent problem solving is that of *planning* the activities of a group of agents. In [Chapter 4](#), we saw how planning could be incorporated as a component of a practical reasoning agent: what extensions or changes might be needed to plan for a team of agents?

Although it is broadly similar in nature to ‘conventional’ planning, of the type seen in [Chapter 4](#), multiagent planning must take into consideration the fact that the activities of agents can interfere with one another – their activities must therefore be coordinated. There are three main possibilities for multiagent planning, as follows [Durfee, 1999, p. 139].

Centralized planning for distributed plans: a centralized planning system develops a plan for a group of agents in which the division and ordering of labour is defined. This ‘master’ agent then distributes the plan to the ‘slaves’, who then execute their part of the plan.

Distributed planning: a group of agents cooperate to form a centralized plan. Typically, the component agents will be ‘specialists’ in different aspects of the overall plan, and will contribute to a part of it. However, the agents that form the plan will not be the ones to

execute it; their role is merely to generate the plan.

Distributed planning for distributed plans: a group of agents *cooperate* to form individual plans of action, dynamically coordinating their activities along the way. The agents may be self-interested, and so, when potential coordination problems arise, they may need to be resolved by negotiation.

In general, centralized planning will be simpler than decentralized planning, because the 'master' can take an overall view, and can dictate coordination relationships as required. The most difficult case to consider is the third. In this case, there may never be a 'global' plan. Individual agents may only ever have pieces of the plan which they are interested in.

Plan merging

[Georgeff, 1983] proposed an algorithm which allows a planner to take a set of plans generated by single agents, and from them generate a conflict free (but not necessarily optimal) multiagent plan. Actions are specified by using a generalization of the STRIPS notation (Chapter 4). In addition to the usual precondition–delete–add lists for actions, Georgeff proposes using a *during* list. This list contains a set of conditions which must hold *while* the action is being carried out. A plan is seen as a set of states; an action is seen as a function which maps the set onto itself. The precondition of an action specifies the domain of the action; the add and delete lists specify the range.

Given a set of single agent plans specified using the modified STRIPS notation, generating a synchronized multiagent plan consists of three stages.

- (1) **Interaction analysis** Interaction analysis involves generating a description of how single agent plans interact with one another. Some of these interactions will be harmless; others will not. Georgeff used the notions of *satisfiability*, *commutativity*, and *precedence* to describe goal interactions. Two actions are said to be *satisfiable* if there is some sequence in which they may be executed without invalidating the preconditions of one or both. *Commutativity* is a restricted case of satisfiability: if two actions may be executed in parallel, then they are said to be commutative. It follows that if two actions are commutative, then either they do not interact, or any interactions are harmless. Precedence describes the sequence in which actions may be executed; if action α_1 has precedence over action α_2 , then the preconditions of α_2 are met by the postconditions of α_1 . That is not to say that α_1 *must* be executed before α_2 ; it is possible for two actions to have precedence over each other.

Interaction analysis involves searching the plans of the agents to detect any interactions between them.

- (2) **Safety analysis** Having determined the possible interactions between plans, it now remains to see which of these interactions are *unsafe*. Georgeff defines safety for pairs of actions in terms of the precedence and commutativity of the pair. Safety analysis involves two stages. First, all actions which are harmless (i.e. where there is no interaction, or the actions commute) are removed from the plan. This is known as simplification. Georgeff shows that the validity of the final plan is not affected by this process, as it is only *boundary* regions that need to be considered. Secondly, the set of all harmful interactions is generated. This stage also involves searching; a rule known as the *commutativity theorem* is applied to reduce the search space. All harmful interactions

the *commutativity theorem* is applied to reduce the search space. All harmful interactions have then been identified.

- (3) **Interaction resolution** In order to resolve conflicts in the simplified plan, Georgeff treats unsafe plan interactions as *critical sections*; to resolve the conflicts, mutual exclusion of the critical sections must be guaranteed. To do this, Georgeff used ideas from Hoare's CSP paradigm to enforce mutual exclusion, although simpler mechanisms (e.g. semaphores) may be used to achieve precisely the same result [Ben-Ari, 1993].

[Stuart, 1985] describes an implemented system which bears a superficial resemblance to Georgeff's algorithm. It takes a set of unsynchronized single agent plans and from them generates a synchronized multiagent plan. Like Georgeff's algorithm, Stuart's system also guarantees a synchronized solution if one exists. Also, the final plan is represented as a sequence of actions interspersed with CSP primitives to guarantee mutual exclusion of critical sections [Hoare, 1978]. Actions are also represented using an extended STRIPS notation. There, however, the resemblance ends. The process of determining which interactions are possibly harmful and resolving conflicts is done not by searching the plans, but by representing the plan as a set of formulae of temporal logic, and attempting to derive a synchronized plan using a temporal logic theorem prover. The idea is that temporal logic is a language for describing sequences of states. As a plan is just a description of exactly such a sequence of states, temporal logic could be used to describe plans. Suppose two plans, π_1 and π_2 , were represented by temporal logic formulae ϕ_1 and ϕ_2 , respectively. Then if the conjunction of these two plans is satisfiable – if there is some sequence of events that is compatible with the conjunction of the formulae – then there is some way that the two plans could be concurrently executed. The temporal logic used was very similar to that used in the Concurrent MetateM language discussed in [Chapter 3](#).

The algorithm to generate a synchronized plan consists of three stages.

1. A set of single agent plans are given as input. They are then translated into a set of formulae in a propositional linear temporal logic (LTL) [Manna and Pnueli, 1992, 1995].
2. The formulae are conjoined and fed into an LTL theorem prover. If the conjoined formula is satisfiable, then the theorem prover will generate a set of sequences of actions which satisfy these formulae. These sequences are encoded in a graph structure. If the formula is not satisfiable, then the theorem prover will report this.
3. The graph generated as output encodes all the possible synchronized executions of the plans. A synchronized plan is then 'read off' from the graph structure.

In general, this approach to multiagent synchronization is computationally expensive, because the temporal theorem prover has to solve a PSPACE-complete problem.

Notes and Further Reading

Published in 1988, Bond and Gasser's *Readings in Distributed Artificial Intelligence* brings together most of the early classic papers on CDPS [Bond and Gasser, 1988]. Although some of the papers that make up this collection are perhaps now rather dated, the survey article written by the editors as a preface to this collection remains one of the most articulate and insightful introductions to the problems and issues of CDPS to date. Victor Lesser and his group at the University of Massachusetts are credited with more or less inventing the field of CDPS, and most innovations in this field to date have originated from

members of this group over the years. Two survey articles that originated from the work of Lesser's group provide overviews of the field: [Durfee et al., [1989a,b](#)]. Another useful survey is [Decker et al., [1989](#)].

The Contract Net has been hugely influential in the multiagent systems literature. It originally formed the basis of Smith's doctoral thesis (published as [Smith, [1980b](#)]), and was further described in [Smith, [1980a](#)] and [Smith and Davis, [1980](#)]. Many variations on the Contract Net theme have been described, including the effectiveness of a Contract Net with 'consultants' that have expertise about the abilities of agents [Tidhar and Rosenschein, [1992](#)], and a sophisticated variation involving marginal cost calculations [Sandholm and Lesser, [1995](#)].

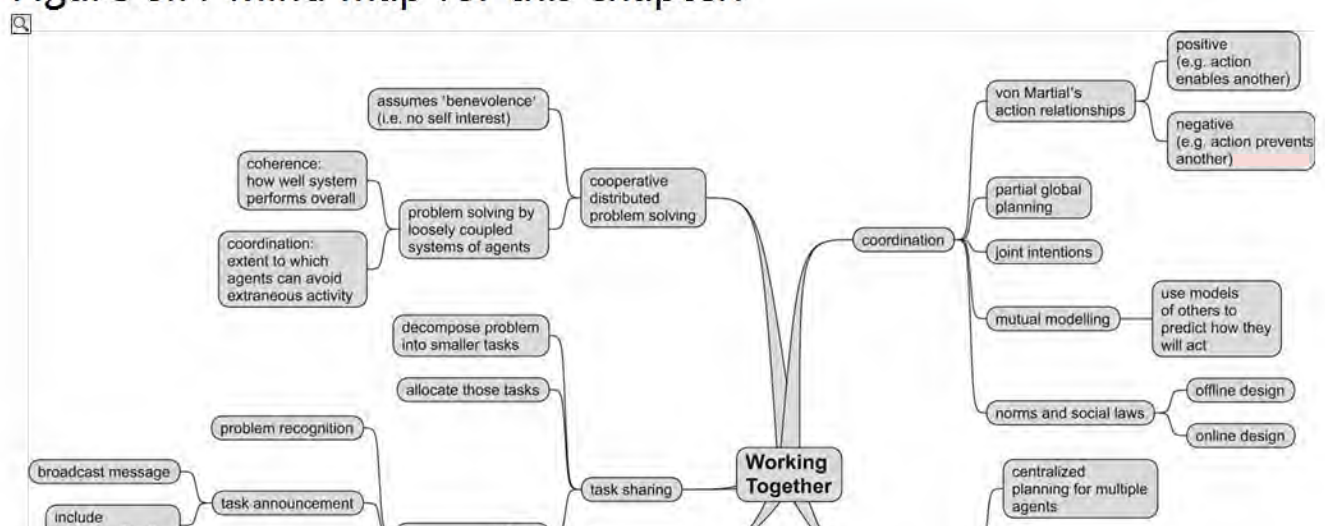
In addition to the model of cooperative action discussed above, a number of other similar formal models of cooperative action have also been developed, the best known of which is probably the Shared Plans model of Barbara Grosz and Sarit Kraus [Grosz and Kraus, [1993](#), [1999](#)].

A number of researchers have considered the development and exploitation of norms and social laws in multiagent systems. Examples of the issues investigated include the control of aggression [Conte and Castelfranchi, [1993](#)], the role of social structure in the emergence of conventions [Kittock, [1993](#)], group behaviour [Findler and Malyankar, [1993](#)], and the reconsideration of commitments [Jennings, [1993a](#)]. In addition, researchers working in philosophy, sociology, and economics have considered similar issues. A good example is the work of [Lewis, [1969](#)], who made some progress towards a (non-formal) theory of normative behaviour.

One issue that I have been forced to omit from this chapter due to space and time limitations is the use of *normative specifications* in multiagent systems, and, in particular, the use of *deontic logic* [Meyer and Wieringa, [1993](#)]. Deontic logic is the logic of obligations and permissions. Originally developed within formal philosophy, deontic logic has been taken up by researchers in computer science in order to express the desirable properties of computer systems. [Dignum, [1999](#)] gives an overview of the use of deontic logic in multiagent systems, and also discusses the general issue of norms and social laws.

Class reading: [Durfee, [1999](#)]. This is a detailed and precise introduction to distributed problem solving and distributed planning, with many useful pointers into the literature.

Figure 8.7: Mind map for this chapter.





¹This is done via a special *report* message type in the original CNET framework.

Chapter 9 Methodologies

As multiagent systems become more established in the collective consciousness of the computer science community, we might expect to see increasing effort devoted to devising *methodologies* to support the development of agent systems. Such methodologies have been highly successful in the object-oriented (OO) community: examples include the methodologies of Booch, and Rumbaugh and colleagues [Booch, 1994; Rumbaugh et al., 1991].

In this chapter I give an overview of work that has been carried out on the development of methodologies for multiagent systems. This work is, at the time of writing, rather tentative – not much experience has yet been gained with them. I begin by considering some of the domain attributes that indicate the appropriateness of an agent-based solution. I then go on to describe various prototypical methodologies, and discuss some of the pitfalls associated with agent-oriented development. I conclude by discussing the technology of *mobile* agents.

9.1 When is an Agent-Based Solution Appropriate?

There are a number of factors which point to the appropriateness of an agent-based approach [Bond and Gasser, 1988; Jennings and Wooldridge, 1998b]:

The environment is open, or at least highly dynamic, uncertain, or complex In such environments, systems capable of flexible autonomous action are often the only solution.

Agents are a natural metaphor Many environments (including most organizations, and any commercial or competitive environment) are naturally modelled as societies of agents, either cooperating with each other to solve complex problems, or else competing with one another. Sometimes, as in intelligent interfaces, the idea of an agent is seen as a natural metaphor: [Maes, 1994a] discusses agents as ‘expert assistants’, cooperating with the user to work on some problem.

Distribution of data, control or expertise In some environments, the distribution of either data, control, or expertise means that a centralized solution is at best extremely difficult or at worst impossible. For example, distributed database systems in which each database is under separate control do not generally lend themselves to centralized solutions. Such systems may often be conveniently modelled as multiagent systems in which each database is a semi-autonomous component.

Legacy systems A problem increasingly faced by software developers is that of *legacy*: software that is technologically obsolete but functionally essential to an organization. Such software cannot generally be discarded, because of the short-term cost of rewriting. And yet it is often required to interact with other software components, which were never imagined by the original designers. One solution to this problem is to *wrap* the legacy components, providing them with an ‘agent layer’ functionality, enabling them to communicate and cooperate with other software components [Genesereth and Ketchpel, 1994].

LEGACY

9.2 Agent-Oriented Analysis and Design

An analysis and design methodology is intended to assist first in gaining an understanding of a particular system, and, secondly, in designing it. Methodologies generally consist of a collection of *models*, and associated with these models, a set of guidelines. The models are intended to formalize understanding of the system being considered. Typically, the models start out as being tentative and rather abstract, and as the analysis and design process continues, they become increasingly more concrete, detailed, and closer to implementation.

Methodologies for the analysis and design of agent-based systems can be broadly divided into two groups:

- those that take their inspiration from object-oriented development, and either extend existing OO methodologies or adapt OO methodologies to the purposes of agent-oriented software engineering (AOSE) [Bauer et al., [2001](#); Burmeister, [1996](#); Depke et al., [2001](#); Kendall, [2001](#); Kinny et al., [1996](#); Odell et al., [2001](#); Omicini, [2001](#); Wood and DeLoach, [2001](#); Wooldridge et al., [1999](#)]
- those that adapt knowledge engineering or other techniques [Brazier et al., [1995](#); Collinot et al., [1996](#); Iglesias et al., [1998](#); Luck et al., [1997](#)].

In the remainder of this section, we review some representative samples of this work.

9.2.1 The AAIL methodology

Throughout the 1990s, the Australian AI Institute (AAIL) developed a range of agent-based systems using their PRS-based belief–desire–intention technology and the distributed multiagent reasoning system (DMARS) [Rao and Georgeff, [1995](#)]. The AAIL methodology for agent-oriented analysis and design was developed as a result of experience gained with these major applications. It draws primarily upon object-oriented methodologies, and enhances them with some agent-based concepts. The methodology itself is aimed at the construction of a set of models which, when fully elaborated, define an agent system specification.

The AAIL methodology provides both *internal* and *external* models. The external model presents a system-level view: the main components visible in this model are agents themselves. The external model is thus primarily concerned with agents and the relationships between them. It is not concerned with the internals of agents: how they are constructed or what they do. In contrast, the internal model is entirely concerned with the internals of agents: their beliefs, desires, and intentions.

INTERNAL AND EXTERNAL MODELS

The external model is intended to define inheritance relationships between agent classes, and to identify the instances of these classes that will appear at run-time. It is itself composed of two further models: the *agent model* and the *interaction model*. The agent model is then further divided into an *agent class model* and an *agent instance model*. These two models define the agents and agent classes that can appear, and relate these classes to one another via inheritance, aggregation, and instantiation relations. Each agent class is assumed to have at least three attributes: beliefs, desires, and intentions. The analyst is able to define how these attributes are overridden during inheritance. For example, it is assumed that, by default, inherited intentions have lower priority than those in subclasses. The analyst may tailor these properties as desired.

AGENT MODEL

INTERACTION MODEL

AGENT CLASS MODEL

AGENT INSTANCE MODEL

Details of the internal model are not given, but it seems clear that developing an internal model corresponds fairly closely to implementing a PRS agent, i.e. designing the agent’s belief, desire, and intention structures.

The AAIL methodology is aimed at elaborating the models described above. It may be summarized as follows.

- 1. Identify the relevant *roles* in the application domain, and, on the basis of these, develop an *agent class hierarchy*. An example role might be a weather monitor, whereby agent *i* is required to make agent *j* aware of the prevailing weather conditions every hour.

ROLES

AGENT CLASS HIERARCHY

- 2. Identify the responsibilities associated with each role, the services required by and provided by the role, and then determine the *goals* associated with each service. With respect to the above example, the goals would be to find out the current weather, and to make agent *j* aware of this information.
- 3. For each goal, determine the plans that may be used to achieve it, and the context conditions under which each plan is appropriate. With respect to the above example, a plan for the goal of making agent *j* aware of the weather conditions might involve sending a message to *j*.
- 4. Determine the belief structure of the system – the information requirements for each plan and goal. With respect to the above example, we might propose a unary predicate *windspeed(x)* to represent the fact that the current wind speed is *x*. A plan to determine the current weather conditions would need to be able to represent this information.

Note that the analysis process will be iterative, as in more traditional methodologies. The outcome will be a model that closely corresponds to the PRS agent architecture. As a result, the move from end-design to implementation using PRS is relatively simple.

Table 9.1: Abstract and concrete concepts in Gaia.

Abstract concepts	Concrete concepts
Roles	Agent types
Permissions	Services
Responsibilities	Acquaintances
Protocols	
Activities	
Liveness properties	